OS CIE 2 - Complete Solved Question Bank

Operating Systems — Fully Solved CIE Question Bank

Units II and III · RV College of Engineering · Exam-style questions and worked answers

Source material used: `unit 2.pdf` (CPU scheduling, scheduling criteria, real-time scheduling) and `unit 3.pdf` (process synchronization, critical sections, Peterson's solution, hardware, semaphores, and classical synchronization problems). All 88 questions in `Pasted markdown.md` are answered, in the same section order. Extended material not directly taught in the slides is explicitly identified.

Conventions for numerical answers: CPU burst times and arrival times are measured in time units unless ms is specified. Lower numerical priority means *higher* priority (Unit 2, p. 39), except where a question explicitly overrides this. Equal arrivals are resolved by process ID; equal shortest bursts by FCFS; no preemption on an exact remaining-time tie. Round Robin admits arrivals occurring at a quantum boundary before re-queuing an unfinished process. Context switches count actual changes from one process to another, excluding initial dispatch and final completion. Scheduling overhead is assumed zero unless given. Where the question is ambiguous, the selected interpretation is stated.

Section 1 — CPU Scheduling: Basic Concepts (12 questions)

Q1. Define CPU scheduling. Explain when CPU scheduling decisions may occur.

Answer:

Definition: CPU scheduling is the operating-system function that chooses a process from the **ready queue** and assigns the CPU to it. It is the basis of multiprogramming: while one process waits for I/O, another can execute so the processor remains productive (Unit 2, pp. 22–23).

The four situations in which a scheduling decision may take place:

1. **Running → Waiting:** a process requests I/O or waits for an event. The CPU becomes available to another ready process.
2. **Running → Ready:** a timer interrupt or higher-priority process preempts the currently running process.

3. **Waiting → Ready:** an I/O operation or event completes; the awakened process may compete for the CPU.
4. **Running → Terminated:** the executing process completes and the CPU must be assigned to another process.

Scheduling in cases 1 and 4 is necessarily non-preemptive (the running process relinquishes the CPU); scheduling decisions in cases 2 and 3 can support preemption. **Goal:** keep the CPU busy and optimize throughput, waiting, turnaround and response times.

Q2. In preemptive scheduling, when can a running process be interrupted?

Answer:

A running process may be interrupted when its **time quantum expires**, when a **higher-priority process becomes ready**, or when a newly arrived process has a **shorter remaining burst** under SRTF. A process can also yield the processor voluntarily or be interrupted by hardware. On preemption: (1) save the running process's registers/program counter in its PCB, (2) move it to the ready state, (3) select another ready process, and (4) restore that process's context. Example: if P1 has remaining CPU time 8 ms and P2 arrives with 2 ms, SRTF preempts P1 to run P2.

Q3. Which algorithm is most suitable for a time-sharing system?

Answer:

Round Robin (RR). It is designed for interactive/time-sharing systems (Unit 2, p. 45). Step 1: maintain the ready queue in FIFO order. Step 2: give the first process a fixed time slice (quantum). Step 3: if it finishes, remove it; otherwise preempt it and append it to the queue. Step 4: repeat. Every waiting process receives frequent CPU access, improving response time. A very small quantum causes context-switch overhead; a very large quantum approaches FCFS.

Q4. Fill in the blanks: _____ scheduling is more appropriate for time-shared/ interactive systems, and _____ scheduling is often used in long-term/job scheduling.

Answer:

Answer: Preemptive scheduling; Non-preemptive scheduling. The first permits interruption for responsive interactive services; the second can finish an admitted batch/job without time slicing. **Note:** A long-term scheduler controls job admission rather than specifically choosing CPU bursts; this is the conventional exam interpretation of the blanks.

Q5. Compare FCFS, SJF, Priority and RR by performance and complexity.

Answer:

Method	Selection rule	Preemptive?	WT / TAT	Response	Starvation / overhead	Complexity of selection
FCFS	Oldest ready arrival	No	Often high; convoy effect	Poor for short jobs behind long job	No starvation if finite jobs; low overhead	O(1) FIFO dequeue
SJF / SRTF	Shortest next burst / shortest remaining	Either	Best average waiting among available jobs when burst lengths known; can reduce TAT	SRTF responds to short new jobs	Long jobs can starve; burst estimation required	O(n) scan, or O(log n) with heap
Priority	Highest priority (smallest number here)	Either	Depends on priority assignment	High-priority tasks respond quickly	Low priorities can starve; aging helps	O(n) scan or O(log n) priority queue
RR	FIFO, bounded time quantum	Yes	Usually above SJF; depends on q	Usually good for interactive use	No starvation under finite ready population; many switches if q small	O(1) queue operations

Conclusion: FCFS is simplest; SJF minimizes average waiting for a fixed set when bursts are known; Priority enforces importance; RR offers fairness and low response delay in time sharing. (Unit 2, pp. 26–29, 33–47.)

Q6. Define the convoy effect and explain its consequences.

Answer:

The **convoy effect** occurs mainly in **FCFS** when a long CPU-bound process runs first and many short I/O-bound processes wait behind it (Unit 2, p. 28).

Example: CPU bursts P1=20, P2=2, P3=2, all ready at time 0. FCFS Gantt: `0–20 P1 | 20–22 P2 | 22–24 P3`. Waiting times are 0, 20 and 22; average = (0+20+22)/3 = **14**. If shorter jobs run first: `0–2 P2 | 2–4 P3 | 4–24 P1`; average WT = (0+2+4)/3 = **2**.

Consequences: larger mean waiting/turnaround, poor response for short tasks, alternation between CPU and I/O device idleness, and reduced effective utilization. **Remedy:** SJF/SRTF or RR where appropriate.

Q7. Explain starvation and aging in CPU scheduling.

Answer:

Starvation (indefinite blocking): a ready process waits for a very long or unlimited time because the scheduling algorithm repeatedly chooses other processes. Example: under strict Priority scheduling, a priority-9 task may never run while priority-1 tasks continuously arrive.

Aging: progressively increase the effective priority of a process the longer it waits. For a scheme where 1 is highest, a waiting process might change from priority 9 → 8 → 7 → ... → 1

at regular intervals. It eventually gets service, preventing starvation. Unit 2, p. 41, identifies starvation as the key Priority-scheduling drawback and aging as the remedy.

Q8. Which scheduling algorithm suffers starvation? Explain the solution with an example.

Answer:

Priority scheduling can cause starvation of low-priority processes; **SJF/SRTF** may also starve long jobs if short jobs keep arriving.

Example: P1 (priority 5) is ready at t=0. Every 2 ms a new process of priority 1 arrives and consumes the CPU for 2 ms. Without intervention P1 can keep waiting indefinitely. Apply **aging:** decrease P1's numerical priority by one after each 4 ms of waiting (5 → 4 → 3 → 2 → 1). Once it reaches the highest-priority class, FCFS within that class eventually schedules P1, provided class membership and aging rules ensure fairness. This restores bounded progress under fair admission.

Q9. State the disadvantages of SJF and Priority scheduling.

Answer:

SJF/SRTF: (1) next CPU burst is not precisely known and must be estimated; (2) large CPU-burst jobs may starve; (3) SRTF creates context switches and monitoring overhead; (4) job length estimation may be inaccurate. **Priority:** (1) low-priority jobs may starve; (2) priority assignment may be subjective or difficult; (3) priority inversion occurs if a low-priority thread holds a lock needed by a high-priority thread; (4) preemptive priority can increase context-switch overhead. **Remedies:** aging for starvation and priority inheritance for priority inversion.

Q10. The time taken by the dispatcher to stop one process and start another is called _____.

Answer:

Dispatch latency. It is the time needed for the dispatcher to stop one running process and start another. A context switch is a principal part of this delay (Unit 2, p. 54).

Q11. Explain the four actions in a context switch with a diagram.

Answer:

Context switch: The CPU stops executing one process and resumes a different process by saving and restoring process state.

Four actions:

1. **Interrupt/trap or scheduling event:** control enters the kernel and a scheduling decision is required.
2. **Save the current context:** store P1's program counter, CPU registers, stack pointer, flags and necessary bookkeeping in P1's PCB; mark P1 as ready or waiting.

3. **Choose the next process and update state:** dispatcher selects P2 from the ready queue and updates its state to running; switch address-space information if needed.
4. **Restore and dispatch:** load P2's saved registers, stack pointer, and program counter and resume its execution.

Diagram:

```
P1 running --(timer / I/O / interrupt)--> kernel
                                           |
                                           v
                                  Save P1 state in PCB1
                                           |
                                   Select ready process P2
                                           |
                                  Restore P2 state from PCB2
                                           v
                                       P2 running
```

Note: CPU work spent switching contexts is overhead, not useful process work; reducing dispatch latency is important for real-time response (Unit 2, p. 54).

Q12. Discuss the different schedulers in operating systems.

Answer:

1. Long-term (job) scheduler: selects which jobs from secondary storage enter memory/ready state. It controls the **degree of multiprogramming** and balances CPU-bound versus I/O-bound jobs. Runs relatively infrequently.

2. Short-term (CPU) scheduler: picks one of the ready processes to execute next. It runs frequently (on interrupts, blocking and termination) and therefore must be fast. FCFS, SJF, Priority and RR are CPU scheduling methods.

3. Medium-term scheduler (swapper): temporarily suspends/swaps out a process from memory and brings it back later to manage memory pressure and multiprogramming.

Flow diagram: `Job pool → [Long-term] → Ready queue → [Short-term] → CPU`; `Ready/blocked ↔ [Medium-term] ↔ Suspended/swapped-out`.

Comparison: long-term = admission; short-term = CPU allocation; medium-term = suspension/resumption.

Section 2 — CPU Scheduling Criteria (4 questions)

Q1. Discuss the criteria for evaluating CPU-scheduling algorithms.

Answer:

Unit 2, pp. 24–25, lists **CPU utilization, throughput, turnaround time, waiting time and response time**.

1. **CPU utilization** = `(CPU busy time / elapsed time) × 100%`; maximize.

2. **Throughput** = `number of processes completed / elapsed time`; maximize.
3. **Turnaround time (TAT)** = `completion time (CT) − arrival time (AT)`; minimize.
4. **Waiting time (WT)** = `TAT − CPU burst time (BT)` for CPU-only numerical problems; minimize. It measures time in the ready queue.
5. **Response time (RT)** = `first time scheduled − arrival time`; minimize, especially in interactive systems.

Example: AT=2, BT=5, first dispatch=6, CT=13 → TAT=13−2=11; WT=11−5=6; RT=6−2=4. Average each metric over all processes for comparisons.

Q2. Define CPU utilization and state its goal.

Answer:

CPU utilization is the percentage of the total observed interval during which the CPU executes useful processes. **Formula:** `CPU utilization (%) = (total CPU busy time / total elapsed time) × 100` (for a problem with scheduling overhead, specify whether overhead counts as useful or occupied time).

Example: Across 100 ms, jobs execute for 75 ms and the CPU is idle for 25 ms. CPU utilization = (75/100)×100 = **75%**. **Goal:** maximize utilization by keeping a ready process executing whenever possible; avoid unnecessary idle time and excessive dispatch overhead (Unit 2, pp. 22, 24–25).

Q3. Compare algorithms on average waiting time and turnaround time.

Answer:

Calculate for each algorithm and the same process workload:

`TAT_i = CT_i − AT_i` and `WT_i = TAT_i − BT_i`; then `Average WT = Σ WT_i / n`, `Average TAT = Σ TAT_i / n`.

Example: Three processes arrive at t=0 with bursts 8, 2, 1 (P1, P2, P3). **FCFS**: `0–8 P1 | 8–10 P2 | 10–11 P3`. WT=(0,8,10), average **6**; TAT=(8,10,11), average **29/3 = 9.67**. **SJF:** `0–1 P3 | 1–3 P2 | 3–11 P1`. WT=(3,1,0), average **4/3 = 1.33**; TAT=(11,3,1), average **15/3 = 5**.

Inference: SJF reduces both averages in this example; in general it has the optimal mean waiting time for a fixed set with known CPU bursts. FCFS may produce a convoy effect. RR usually sacrifices some mean WT/TAT for lower response time and fairness. Priority varies with assigned priorities.

Q4. Explain the effect of context switches on scheduling performance.

Answer:

A **context switch** saves the current process state and restores another process state. It takes CPU time but performs no useful application processing. If N context switches each cost d ms, total switching overhead is approximately `N × d` ms. Example: 40 switches × 0.2 ms = **8 ms** lost to switching.

Impacts: (1) reduces effective throughput and CPU time available for work, (2) increases completion/turnaround times, (3) can harm cache/TLB locality, and (4) too many switches increase scheduler overhead. **RR trade-off:** very small q produces many switches and fast initial response; very large q approaches FCFS (Unit 2, pp. 47–48). **Good design:** choose a practical quantum, use efficient dispatching, and avoid unnecessary preemption.

Section 3 — Scheduling Algorithms: Fully Worked Numerical Problems (19 questions)

Standard method for every numerical: (1) list AT and BT; (2) select processes according to the algorithm and show each CPU interval; (3) determine CT from the Gantt chart; (4) compute TAT = CT − AT; (5) compute WT = TAT − BT; (6) if needed, RT = first start − AT; (7) sum each metric and divide by the number of processes. Additional execution-trace tables display the time progress, not just final answers.

Q1. Calculate FCFS and SJF average WT/TAT for P1(0,10), P2(2,5), P3(4,8), P4(6,2).

Answer — Given process data:

Process	Arrival (AT)	Burst (BT)
P1	0	10
P2	2	5
P3	4	8
P4	6	2

FCFS

Selection rule: Earliest arrival first, without preemption; use process-ID order for an equal-arrival tie.

Gantt chart (time interval : process):

```
0-10 : P1  |  10-15 : P2  |  15-23 : P3  |  23-25 : P4
```

Execution trace (every dispatch interval):

Interval	CPU / process	Duration	Remaining burst after interval
0–10	P1	10	0
10–15	P2	5	0
15–23	P3	8	0
23–25	P4	2	0

Calculation formulas: CT = completion time; TAT = CT − AT; WT = TAT − BT; RT = first CPU start − AT.

Process-wise calculations:

PID	AT	BT	CT	TAT = CT−AT	WT = TAT−BT	RT
P1	0	10	10	10−0 = **10**	10−10 = **0**	0
P2	2	5	15	15−2 = **13**	13−5 = **8**	8
P3	4	8	23	23−4 = **19**	19−8 = **11**	11
P4	6	2	25	25−6 = **19**	19−2 = **17**	17

Averages:

- Average WT = (0 + 8 + 11 + 17) / 4 = 36 / 4 = **9**.
- Average TAT = (10 + 13 + 19 + 19) / 4 = 61 / 4 = **15.25**.
- Average RT = (0 + 8 + 11 + 17) / 4 = 36 / 4 = **9**.
- Context switches (between different processes; do not count initial dispatch or final termination) = **3**.

SJF (non-preemptive)

Selection rule: At each CPU release, pick the available process with the smallest CPU burst and run it to completion.

Gantt chart (time interval : process):

```
0-10 : P1  |  10-12 : P4  |  12-17 : P2  |  17-25 : P3
```

Execution trace (every dispatch interval):

Interval	CPU / process	Duration	Remaining burst after interval
0–10	P1	10	0
10–12	P4	2	0
12–17	P2	5	0
17–25	P3	8	0

Calculation formulas: CT = completion time; TAT = CT − AT; WT = TAT − BT; RT = first CPU start − AT.

Process-wise calculations:

PID	AT	BT	CT	TAT = CT−AT	WT = TAT−BT	RT
P1	0	10	10	10−0 = **10**	10−10 = **0**	0
P2	2	5	17	17−2 = **15**	15−5 = **10**	10
P3	4	8	25	25−4 = **21**	21−8 = **13**	13
P4	6	2	12	12−6 = **6**	6−2 = **4**	4

Averages:

- Average WT = (0 + 10 + 13 + 4) / 4 = 27 / 4 = **6.75**.
- Average TAT = (10 + 15 + 21 + 6) / 4 = 52 / 4 = **13**.
- Average RT = (0 + 10 + 13 + 4) / 4 = 27 / 4 = **6.75**.
- Context switches (between different processes; do not count initial dispatch or final termination) = **3**.

Comparison: FCFS average WT = 9, TAT = 15.25; SJF average WT = 6.75, TAT = 13. SJF saves **2.25 waiting-time units** and **2.25 turnaround-time units** by selecting P4 before longer waiting jobs at t=10.

Q2. Using non-preemptive SJF, calculate Gantt chart, average WT and average TAT for the five specified processes.

Answer — Given process data:

Process	Arrival (AT)	Burst (BT)
P1	3	1
P2	1	4
P3	4	2
P4	0	6
P5	2	3

SJF (non-preemptive)

Selection rule: At each CPU release, pick the available process with the smallest CPU burst and run it to completion.

Gantt chart (time interval : process):

```
0-6 : P4  |  6-7 : P1  |  7-9 : P3  |  9-12 : P5  |  12-16 : P2
```

Execution trace (every dispatch interval):

Interval	CPU / process	Duration	Remaining burst after interval
0–6	P4	6	0
6–7	P1	1	0
7–9	P3	2	0
9–12	P5	3	0
12–16	P2	4	0

Calculation formulas: CT = completion time; TAT = CT − AT; WT = TAT − BT; RT = first CPU start − AT.

Process-wise calculations:

PID	AT	BT	CT	TAT = CT−AT	WT = TAT−BT	RT
P1	3	1	7	7−3 = **4**	4−1 = **3**	3
P2	1	4	16	16−1 = **15**	15−4 = **11**	11
P3	4	2	9	9−4 = **5**	5−2 = **3**	3
P4	0	6	6	6−0 = **6**	6−6 = **0**	0
P5	2	3	12	12−2 = **10**	10−3 = **7**	7

Averages:

- Average WT = (3 + 11 + 3 + 0 + 7) / 5 = 24 / 5 = **4.8**.
- Average TAT = (4 + 15 + 5 + 6 + 10) / 5 = 40 / 5 = **8**.
- Average RT = (3 + 11 + 3 + 0 + 7) / 5 = 24 / 5 = **4.8**.
- Context switches (between different processes; do not count initial dispatch or final termination) = **4**.

Q3. Solve FCFS and analyze the effect of changing the arrival times (convoy effect).

Answer — Given process data:

Process	Arrival (AT)	Burst (BT)
P1	1	1
P2	1	3
P3	0	40

FCFS

Selection rule: Earliest arrival first, without preemption; use process-ID order for an equal-arrival tie.

Gantt chart (time interval : process):

```
0-40 : P3  |  40-41 : P1  |  41-44 : P2
```

Execution trace (every dispatch interval):

Interval	CPU / process	Duration	Remaining burst after interval
0–40	P3	40	0
40–41	P1	1	0
41–44	P2	3	0

Calculation formulas: CT = completion time; TAT = CT − AT; WT = TAT − BT; RT = first CPU start − AT.

Process-wise calculations:

PID	AT	BT	CT	TAT = CT−AT	WT = TAT−BT	RT
P1	1	1	41	41−1 = **40**	40−1 = **39**	39
P2	1	3	44	44−1 = **43**	43−3 = **40**	40
P3	0	40	40	40−0 = **40**	40−40 = **0**	0

Averages:

- Average WT = (39 + 40 + 0) / 3 = 79 / 3 = **26.3333**.
- Average TAT = (40 + 43 + 40) / 3 = 123 / 3 = **41**.
- Average RT = (39 + 40 + 0) / 3 = 79 / 3 = **26.3333**.
- Context switches (between different processes; do not count initial dispatch or final termination) = **2**.

Changed arrivals: P1=0, P2=0, P3=1 (bursts unchanged)

FCFS

Selection rule: Earliest arrival first, without preemption; use process-ID order for an equal-arrival tie.

Gantt chart (time interval : process):

```
0-1 : P1  |  1-4 : P2  |  4-44 : P3
```

Execution trace (every dispatch interval):

Interval	CPU / process	Duration	Remaining burst after interval
0–1	P1	1	0
1–4	P2	3	0
4–44	P3	40	0

Calculation formulas: CT = completion time; TAT = CT − AT; WT = TAT − BT; RT = first CPU start − AT.

Process-wise calculations:

PID	AT	BT	CT	TAT = CT−AT	WT = TAT−BT	RT
P1	0	1	1	1−0 = **1**	1−1 = **0**	0
P2	0	3	4	4−0 = **4**	4−3 = **1**	1
P3	1	40	44	44−1 = **43**	43−40 = **3**	3

Averages:

- Average WT = (0 + 1 + 3) / 3 = 4 / 3 = **1.3333**.
- Average TAT = (1 + 4 + 43) / 3 = 48 / 3 = **16**.
- Average RT = (0 + 1 + 3) / 3 = 4 / 3 = **1.3333**.

- Context switches (between different processes; do not count initial dispatch or final termination) = **2**.

Variation: average WT changes from 26.3333 to 1.3333 (decrease 25); average TAT changes from 41 to 16 (decrease 25). **Yes**, the convoy effect clearly appears in the first arrangement, where the 40-unit P3 runs before the two short jobs. It is substantially alleviated in the modified arrival order.

Q4. Draw Gantt charts and calculate WT/TAT for SJF, preemptive Priority and RR (q=1); find the maximum ready-queue occupancy.

Answer — Given process data:

Process	Arrival (AT)	Burst (BT)	Priority
P1	0	7	3
P2	3	2	2
P3	4	3	1
P4	4	1	1
P5	5	3	3

Interpretation: The unspecified SJF variant in Q4 is treated as **non-preemptive SJF**; Priority is **preemptive**, as stated.

SJF (non-preemptive)

Selection rule: At each CPU release, pick the available process with the smallest CPU burst and run it to completion.

Gantt chart (time interval : process):

```
0-7 : P1  |  7-8 : P4  |  8-10 : P2  |  10-13 : P3  |  13-16 : P5
```

Execution trace (every dispatch interval):

Interval	CPU / process	Duration	Remaining burst after interval
0–7	P1	7	0
7–8	P4	1	0
8–10	P2	2	0
10–13	P3	3	0
13–16	P5	3	0

Calculation formulas: CT = completion time; TAT = CT − AT; WT = TAT − BT; RT = first CPU start − AT.

Process-wise calculations:

PID	AT	BT	CT	TAT = CT−AT	WT = TAT−BT	RT
P1	0	7	7	7−0 = **7**	7−7 = **0**	0
P2	3	2	10	10−3 = **7**	7−2 = **5**	5
P3	4	3	13	13−4 = **9**	9−3 = **6**	6
P4	4	1	8	8−4 = **4**	4−1 = **3**	3
P5	5	3	16	16−5 = **11**	11−3 = **8**	8

Averages:

- Average WT = (0 + 5 + 6 + 3 + 8) / 5 = 22 / 5 = **4.4**.
- Average TAT = (7 + 7 + 9 + 4 + 11) / 5 = 38 / 5 = **7.6**.
- Average RT = (0 + 5 + 6 + 3 + 8) / 5 = 22 / 5 = **4.4**.
- Context switches (between different processes; do not count initial dispatch or final termination) = **4**.

Priority (preemptive)

Selection rule: Choose the smallest numerical priority value; a strictly higher-priority arrival preempts. Equal-priority requests use FCFS order.

Gantt chart (time interval : process):

`0-3 : P1 | 3-4 : P2 | 4-7 : P3 | 7-8 : P4 | 8-9 : P2 | 9-13 : P1`

`13-16 : P5`

Execution trace (every dispatch interval):

Interval	CPU / process	Duration	Remaining burst after interval
0–3	P1	3	4
3–4	P2	1	1
4–7	P3	3	0
7–8	P4	1	0
8–9	P2	1	0
9–13	P1	4	0
13–16	P5	3	0

Calculation formulas: CT = completion time; TAT = CT − AT; WT = TAT − BT; RT = first CPU start − AT.

Process-wise calculations:

PID	AT	BT	CT	TAT = CT−AT	WT = TAT−BT	RT
P1	0	7	13	13−0 = **13**	13−7 = **6**	0
P2	3	2	9	9−3 = **6**	6−2 = **4**	0

PID	AT	BT	CT	TAT = CT−AT	WT = TAT−BT	RT
P3	4	3	7	7−4 = **3**	3−3 = **0**	0
P4	4	1	8	8−4 = **4**	4−1 = **3**	3
P5	5	3	16	16−5 = **11**	11−3 = **8**	8

Averages:

- Average WT = (6 + 4 + 0 + 3 + 8) / 5 = 21 / 5 = **4.2**.
- Average TAT = (13 + 6 + 3 + 4 + 11) / 5 = 37 / 5 = **7.4**.
- Average RT = (0 + 0 + 0 + 3 + 8) / 5 = 11 / 5 = **2.2**.
- Context switches (between different processes; do not count initial dispatch or final termination) = **6**.

Round Robin, q = 1

Selection rule: Each ready process receives at most 1 time units; if unfinished, it returns to the back of the ready queue. Arrivals occurring exactly when the slice ends are entered before re-queuing the unfinished process.

Gantt chart (time interval : process):

```
0-1 : P1  |  1-2 : P1  |  2-3 : P1  |  3-4 : P2  |  4-5 : P1  |  5-6 : P3
```

```
6-7 : P4  |  7-8 : P2  |  8-9 : P5  |  9-10 : P1  |  10-11 : P3  |  11-12 : P5
```

```
12-13 : P1  |  13-14 : P3  |  14-15 : P5  |  15-16 : P1
```

Execution trace (every dispatch interval):

Interval	CPU / process	Duration	Remaining burst after interval
0–1	P1	1	6
1–2	P1	1	5
2–3	P1	1	4
3–4	P2	1	1
4–5	P1	1	3
5–6	P3	1	2
6–7	P4	1	0
7–8	P2	1	0
8–9	P5	1	2
9–10	P1	1	2
10–11	P3	1	1
11–12	P5	1	1
12–13	P1	1	1

Interval	CPU / process	Duration	Remaining burst after interval
13–14	P3	1	0
14–15	P5	1	0
15–16	P1	1	0

Calculation formulas: CT = completion time; TAT = CT − AT; WT = TAT − BT; RT = first CPU start − AT.

Process-wise calculations:

PID	AT	BT	CT	TAT = CT−AT	WT = TAT−BT	RT
P1	0	7	16	16−0 = **16**	16−7 = **9**	0
P2	3	2	8	8−3 = **5**	5−2 = **3**	0
P3	4	3	14	14−4 = **10**	10−3 = **7**	1
P4	4	1	7	7−4 = **3**	3−1 = **2**	2
P5	5	3	15	15−5 = **10**	10−3 = **7**	3

Averages:

- Average WT = (9 + 3 + 7 + 2 + 7) / 5 = 28 / 5 = **5.6**.
- Average TAT = (16 + 5 + 10 + 3 + 10) / 5 = 44 / 5 = **8.8**.
- Average RT = (0 + 0 + 1 + 2 + 3) / 5 = 6 / 5 = **1.2**.
- Context switches (between different processes; do not count initial dispatch or final termination) = **13**.

Minimum: pprio has the smallest average waiting time (4.2); pprio has the smallest average turnaround time (7.4).

Ready-queue maximum: At **t = 5**, all five processes have arrived, and one occupies the CPU while **four wait in the ready queue** (P1–P5 excluding the running process). Therefore maximum ready-queue population is **4**. Counting the running process too, maximum number in the system is **5**. This maximum occurs across the three schedules, though the identities differ.

Q5. Compare FCFS and preemptive SJF for fractional arrival/burst times.

Answer — Given process data:

Process	Arrival (AT)	Burst (BT)
P1	0	1.5
P2	1.5	3
P3	3	1
P4	3	7.5

FCFS

Selection rule: Earliest arrival first, without preemption; use process-ID order for an equal-arrival tie.

Gantt chart (time interval : process):

`0-1.5 : P1 | 1.5-4.5 : P2 | 4.5-5.5 : P3 | 5.5-13 : P4`

Execution trace (every dispatch interval):

Interval	CPU / process	Duration	Remaining burst after interval
0–1.5	P1	1.5	0
1.5–4.5	P2	3	0
4.5–5.5	P3	1	0
5.5–13	P4	7.5	0

Calculation formulas: CT = completion time; TAT = CT − AT; WT = TAT − BT; RT = first CPU start − AT.

Process-wise calculations:

PID	AT	BT	CT	TAT = CT−AT	WT = TAT−BT	RT
P1	0	1.5	1.5	1.5−0 = **1.5**	1.5−1.5 = **0**	0
P2	1.5	3	4.5	4.5−1.5 = **3**	3−3 = **0**	0
P3	3	1	5.5	5.5−3 = **2.5**	2.5−1 = **1.5**	1.5
P4	3	7.5	13	13−3 = **10**	10−7.5 = **2.5**	2.5

Averages:

- Average WT = (0 + 0 + 1.5 + 2.5) / 4 = 4 / 4 = **1**.
- Average TAT = (1.5 + 3 + 2.5 + 10) / 4 = 17 / 4 = **4.25**.
- Average RT = (0 + 0 + 1.5 + 2.5) / 4 = 4 / 4 = **1**.
- Context switches (between different processes; do not count initial dispatch or final termination) = **3**.

SJF (preemptive) / SRTF

Selection rule: At every arrival or completion, choose the available process with the shortest remaining time; retain the running process on an exact tie.

Gantt chart (time interval : process):

`0-1.5 : P1 | 1.5-3 : P2 | 3-4 : P3 | 4-5.5 : P2 | 5.5-13 : P4`

Execution trace (every dispatch interval):

Interval	CPU / process	Duration	Remaining burst after interval
0–1.5	P1	1.5	0

Interval	CPU / process	Duration	Remaining burst after interval
1.5–3	P2	1.5	1.5
3–4	P3	1	0
4–5.5	P2	1.5	0
5.5–13	P4	7.5	0

Calculation formulas: CT = completion time; TAT = CT − AT; WT = TAT − BT; RT = first CPU start − AT.

Process-wise calculations:

PID	AT	BT	CT	TAT = CT−AT	WT = TAT−BT	RT
P1	0	1.5	1.5	1.5−0 = **1.5**	1.5−1.5 = **0**	0
P2	1.5	3	5.5	5.5−1.5 = **4**	4−3 = **1**	0
P3	3	1	4	4−3 = **1**	1−1 = **0**	0
P4	3	7.5	13	13−3 = **10**	10−7.5 = **2.5**	2.5

Averages:

- Average WT = (0 + 1 + 0 + 2.5) / 4 = 3.5 / 4 = **0.875**.
- Average TAT = (1.5 + 4 + 1 + 10) / 4 = 16.5 / 4 = **4.125**.
- Average RT = (0 + 0 + 0 + 2.5) / 4 = 2.5 / 4 = **0.625**.
- Context switches (between different processes; do not count initial dispatch or final termination) = **4**.

Q6. Solve FCFS and preemptive SJF for P1(0,6), P2(4,2), P3(3,7), P4(3,1).

Answer — Given process data:

Process	Arrival (AT)	Burst (BT)
P1	0	6
P2	4	2
P3	3	7
P4	3	1

FCFS

Selection rule: Earliest arrival first, without preemption; use process-ID order for an equal-arrival tie.

Gantt chart (time interval : process):

```
0-6 : P1  |  6-13 : P3  |  13-14 : P4  |  14-16 : P2
```

Execution trace (every dispatch interval):

Interval	CPU / process	Duration	Remaining burst after interval
0–6	P1	6	0
6–13	P3	7	0
13–14	P4	1	0
14–16	P2	2	0

Calculation formulas: CT = completion time; TAT = CT − AT; WT = TAT − BT; RT = first CPU start − AT.

Process-wise calculations:

PID	AT	BT	CT	TAT = CT−AT	WT = TAT−BT	RT
P1	0	6	6	6−0 = **6**	6−6 = **0**	0
P2	4	2	16	16−4 = **12**	12−2 = **10**	10
P3	3	7	13	13−3 = **10**	10−7 = **3**	3
P4	3	1	14	14−3 = **11**	11−1 = **10**	10

Averages:

- Average WT = (0 + 10 + 3 + 10) / 4 = 23 / 4 = **5.75**.
- Average TAT = (6 + 12 + 10 + 11) / 4 = 39 / 4 = **9.75**.
- Average RT = (0 + 10 + 3 + 10) / 4 = 23 / 4 = **5.75**.
- Context switches (between different processes; do not count initial dispatch or final termination) = **3**.

SJF (preemptive) / SRTF

Selection rule: At every arrival or completion, choose the available process with the shortest remaining time; retain the running process on an exact tie.

Gantt chart (time interval : process):

```
0-3 : P1  |  3-4 : P4  |  4-6 : P2  |  6-9 : P1  |  9-16 : P3
```

Execution trace (every dispatch interval):

Interval	CPU / process	Duration	Remaining burst after interval
0–3	P1	3	3
3–4	P4	1	0
4–6	P2	2	0
6–9	P1	3	0
9–16	P3	7	0

Calculation formulas: CT = completion time; TAT = CT − AT; WT = TAT − BT; RT = first CPU start − AT.

Process-wise calculations:

PID	AT	BT	CT	TAT = CT−AT	WT = TAT−BT	RT
P1	0	6	9	9−0 = **9**	9−6 = **3**	0
P2	4	2	6	6−4 = **2**	2−2 = **0**	0
P3	3	7	16	16−3 = **13**	13−7 = **6**	6
P4	3	1	4	4−3 = **1**	1−1 = **0**	0

Averages:

- Average WT = (3 + 0 + 6 + 0) / 4 = 9 / 4 = **2.25**.
- Average TAT = (9 + 2 + 13 + 1) / 4 = 25 / 4 = **6.25**.
- Average RT = (0 + 0 + 6 + 0) / 4 = 6 / 4 = **1.5**.
- Context switches (between different processes; do not count initial dispatch or final termination) = **4**.

Q7. Solve preemptive SJF, Priority and RR (q=1): Gantt charts, WT/TAT and context switches.

Answer — Given process data:

Process	Arrival (AT)	Burst (BT)	Priority
P1	0	10	3
P2	3	5	2
P3	4	8	1
P4	4	2	1
P5	5	12	2

Interpretation: The question just says “Priority,” so **preemptive Priority** is used, matching the Unit-2 priority scheduling worked example (p. 42). If your teacher asks for non-preemptive Priority instead, the Gantt chart and means will differ.

SJF (preemptive) / SRTF

Selection rule: At every arrival or completion, choose the available process with the shortest remaining time; retain the running process on an exact tie.

Gantt chart (time interval : process):

```
0-3 : P1  |  3-4 : P2  |  4-6 : P4  |  6-10 : P2  |  10-17 : P1  |  17-25 : P3

25-37 : P5
```

Execution trace (every dispatch interval):

Interval	CPU / process	Duration	Remaining burst after interval
0–3	P1	3	7

Interval	CPU / process	Duration	Remaining burst after interval
3–4	P2	1	4
4–6	P4	2	0
6–10	P2	4	0
10–17	P1	7	0
17–25	P3	8	0
25–37	P5	12	0

Calculation formulas: CT = completion time; TAT = CT − AT; WT = TAT − BT; RT = first CPU start − AT.

Process-wise calculations:

PID	AT	BT	CT	TAT = CT−AT	WT = TAT−BT	RT
P1	0	10	17	17−0 = **17**	17−10 = **7**	0
P2	3	5	10	10−3 = **7**	7−5 = **2**	0
P3	4	8	25	25−4 = **21**	21−8 = **13**	13
P4	4	2	6	6−4 = **2**	2−2 = **0**	0
P5	5	12	37	37−5 = **32**	32−12 = **20**	20

Averages:

- Average WT = (7 + 2 + 13 + 0 + 20) / 5 = 42 / 5 = **8.4**.
- Average TAT = (17 + 7 + 21 + 2 + 32) / 5 = 79 / 5 = **15.8**.
- Average RT = (0 + 0 + 13 + 0 + 20) / 5 = 33 / 5 = **6.6**.
- Context switches (between different processes; do not count initial dispatch or final termination) = **6**.

Priority (preemptive)

Selection rule: Choose the smallest numerical priority value; a strictly higher-priority arrival preempts. Equal-priority requests use FCFS order.

Gantt chart (time interval : process):

```
0-3 : P1  |  3-4 : P2  |  4-12 : P3  |  12-14 : P4  |  14-18 : P2  |  18-30 : P5
30-37 : P1
```

Execution trace (every dispatch interval):

Interval	CPU / process	Duration	Remaining burst after interval
0–3	P1	3	7
3–4	P2	1	4
4–12	P3	8	0

Interval	CPU / process	Duration	Remaining burst after interval
12–14	P4	2	0
14–18	P2	4	0
18–30	P5	12	0
30–37	P1	7	0

Calculation formulas: CT = completion time; TAT = CT − AT; WT = TAT − BT; RT = first CPU start − AT.

Process-wise calculations:

PID	AT	BT	CT	TAT = CT−AT	WT = TAT−BT	RT
P1	0	10	37	37−0 = **37**	37−10 = **27**	0
P2	3	5	18	18−3 = **15**	15−5 = **10**	0
P3	4	8	12	12−4 = **8**	8−8 = **0**	0
P4	4	2	14	14−4 = **10**	10−2 = **8**	8
P5	5	12	30	30−5 = **25**	25−12 = **13**	13

Averages:

- Average WT = (27 + 10 + 0 + 8 + 13) / 5 = 58 / 5 = **11.6**.
- Average TAT = (37 + 15 + 8 + 10 + 25) / 5 = 95 / 5 = **19**.
- Average RT = (0 + 0 + 0 + 8 + 13) / 5 = 21 / 5 = **4.2**.
- Context switches (between different processes; do not count initial dispatch or final termination) = **6**.

Round Robin, q = 1

Selection rule: Each ready process receives at most 1 time units; if unfinished, it returns to the back of the ready queue. Arrivals occurring exactly when the slice ends are entered before re-queuing the unfinished process.

Gantt chart (time interval : process):

```
0-1 : P1  |  1-2 : P1  |  2-3 : P1  |  3-4 : P2  |  4-5 : P1  |  5-6 : P3

6-7 : P4  |  7-8 : P2  |  8-9 : P5  |  9-10 : P1  |  10-11 : P3  |  11-12 : P4

12-13 : P2  |  13-14 : P5  |  14-15 : P1  |  15-16 : P3  |  16-17 : P2  |  17-18 : P5

18-19 : P1  |  19-20 : P3  |  20-21 : P2  |  21-22 : P5  |  22-23 : P1  |  23-24 : P3

24-25 : P5  |  25-26 : P1  |  26-27 : P3  |  27-28 : P5  |  28-29 : P1  |  29-30 : P3

30-31 : P5  |  31-32 : P3  |  32-33 : P5  |  33-34 : P5  |  34-35 : P5  |  35-36 : P5

36-37 : P5
```

Execution trace (every dispatch interval):

Interval	CPU / process	Duration	Remaining burst after interval
0–1	P1	1	9
1–2	P1	1	8
2–3	P1	1	7
3–4	P2	1	4
4–5	P1	1	6
5–6	P3	1	7
6–7	P4	1	1
7–8	P2	1	3
8–9	P5	1	11
9–10	P1	1	5
10–11	P3	1	6
11–12	P4	1	0
12–13	P2	1	2
13–14	P5	1	10
14–15	P1	1	4
15–16	P3	1	5
16–17	P2	1	1
17–18	P5	1	9
18–19	P1	1	3
19–20	P3	1	4
20–21	P2	1	0
21–22	P5	1	8
22–23	P1	1	2
23–24	P3	1	3
24–25	P5	1	7
25–26	P1	1	1
26–27	P3	1	2
27–28	P5	1	6
28–29	P1	1	0
29–30	P3	1	1
30–31	P5	1	5
31–32	P3	1	0
32–33	P5	1	4

Interval	CPU / process	Duration	Remaining burst after interval
33–34	P5	1	3
34–35	P5	1	2
35–36	P5	1	1
36–37	P5	1	0

Calculation formulas: CT = completion time; TAT = CT − AT; WT = TAT − BT; RT = first CPU start − AT.

Process-wise calculations:

PID	AT	BT	CT	TAT = CT−AT	WT = TAT−BT	RT
P1	0	10	29	29−0 = **29**	29−10 = **19**	0
P2	3	5	21	21−3 = **18**	18−5 = **13**	0
P3	4	8	32	32−4 = **28**	28−8 = **20**	1
P4	4	2	12	12−4 = **8**	8−2 = **6**	2
P5	5	12	37	37−5 = **32**	32−12 = **20**	3

Averages:

- Average WT = (19 + 13 + 20 + 6 + 20) / 5 = 78 / 5 = **15.6**.
- Average TAT = (29 + 18 + 28 + 8 + 32) / 5 = 115 / 5 = **23**.
- Average RT = (0 + 0 + 1 + 2 + 3) / 5 = 6 / 5 = **1.2**.
- Context switches (between different processes; do not count initial dispatch or final termination) = **30**.

Context-switch method: count a switch whenever consecutive execution intervals have **different** PIDs, excluding initial CPU dispatch and final termination; no switch is added for an uninterrupted continuation of the same PID.

Q8. All processes arrive at zero; solve FCFS, SJF, non-preemptive Priority and RR (q=1).

Answer — Given process data:

Process	Arrival (AT)	Burst (BT)	Priority
P1	0	10	3
P2	0	1	1
P3	0	2	3
P4	0	1	4
P5	0	5	2

All processes arrive in the order P1, P2, P3, P4, P5 at t=0; priority **1 is highest**. In Q8 the question explicitly requests **non-preemptive** priority.

FCFS

Selection rule: Earliest arrival first, without preemption; use process-ID order for an equal-arrival tie.

Gantt chart (time interval : process):

```
0-10 : P1  |  10-11 : P2  |  11-13 : P3  |  13-14 : P4  |  14-19 : P5
```

Execution trace (every dispatch interval):

Interval	CPU / process	Duration	Remaining burst after interval
0–10	P1	10	0
10–11	P2	1	0
11–13	P3	2	0
13–14	P4	1	0
14–19	P5	5	0

Calculation formulas: CT = completion time; TAT = CT − AT; WT = TAT − BT; RT = first CPU start − AT.

Process-wise calculations:

PID	AT	BT	CT	TAT = CT−AT	WT = TAT−BT	RT
P1	0	10	10	10−0 = **10**	10−10 = **0**	0
P2	0	1	11	11−0 = **11**	11−1 = **10**	10
P3	0	2	13	13−0 = **13**	13−2 = **11**	11
P4	0	1	14	14−0 = **14**	14−1 = **13**	13
P5	0	5	19	19−0 = **19**	19−5 = **14**	14

Averages:

- Average WT = (0 + 10 + 11 + 13 + 14) / 5 = 48 / 5 = **9.6**.
- Average TAT = (10 + 11 + 13 + 14 + 19) / 5 = 67 / 5 = **13.4**.
- Average RT = (0 + 10 + 11 + 13 + 14) / 5 = 48 / 5 = **9.6**.
- Context switches (between different processes; do not count initial dispatch or final termination) = **4**.

SJF (non-preemptive)

Selection rule: At each CPU release, pick the available process with the smallest CPU burst and run it to completion.

Gantt chart (time interval : process):

```
0-1 : P2  |  1-2 : P4  |  2-4 : P3  |  4-9 : P5  |  9-19 : P1
```

Execution trace (every dispatch interval):

Interval	CPU / process	Duration	Remaining burst after interval
0–1	P2	1	0
1–2	P4	1	0
2–4	P3	2	0
4–9	P5	5	0
9–19	P1	10	0

Calculation formulas: CT = completion time; TAT = CT − AT; WT = TAT − BT; RT = first CPU start − AT.

Process-wise calculations:

PID	AT	BT	CT	TAT = CT−AT	WT = TAT−BT	RT
P1	0	10	19	19−0 = **19**	19−10 = **9**	9
P2	0	1	1	1−0 = **1**	1−1 = **0**	0
P3	0	2	4	4−0 = **4**	4−2 = **2**	2
P4	0	1	2	2−0 = **2**	2−1 = **1**	1
P5	0	5	9	9−0 = **9**	9−5 = **4**	4

Averages:

- Average WT = (9 + 0 + 2 + 1 + 4) / 5 = 16 / 5 = **3.2**.
- Average TAT = (19 + 1 + 4 + 2 + 9) / 5 = 35 / 5 = **7**.
- Average RT = (9 + 0 + 2 + 1 + 4) / 5 = 16 / 5 = **3.2**.
- Context switches (between different processes; do not count initial dispatch or final termination) = **4**.

Priority (non-preemptive)

Selection rule: Choose the smallest priority number; once selected, finish the burst. Equal-priority processes use FCFS.

Gantt chart (time interval : process):

```
0-1 : P2  |  1-6 : P5  |  6-16 : P1  |  16-18 : P3  |  18-19 : P4
```

Execution trace (every dispatch interval):

Interval	CPU / process	Duration	Remaining burst after interval
0–1	P2	1	0
1–6	P5	5	0
6–16	P1	10	0

Interval	CPU / process	Duration	Remaining burst after interval
16–18	P3	2	0
18–19	P4	1	0

Calculation formulas: CT = completion time; TAT = CT − AT; WT = TAT − BT; RT = first CPU start − AT.

Process-wise calculations:

PID	AT	BT	CT	TAT = CT−AT	WT = TAT−BT	RT
P1	0	10	16	16−0 = **16**	16−10 = **6**	6
P2	0	1	1	1−0 = **1**	1−1 = **0**	0
P3	0	2	18	18−0 = **18**	18−2 = **16**	16
P4	0	1	19	19−0 = **19**	19−1 = **18**	18
P5	0	5	6	6−0 = **6**	6−5 = **1**	1

Averages:

- Average WT = (6 + 0 + 16 + 18 + 1) / 5 = 41 / 5 = **8.2**.
- Average TAT = (16 + 1 + 18 + 19 + 6) / 5 = 60 / 5 = **12**.
- Average RT = (6 + 0 + 16 + 18 + 1) / 5 = 41 / 5 = **8.2**.
- Context switches (between different processes; do not count initial dispatch or final termination) = **4**.

Round Robin, q = 1

Selection rule: Each ready process receives at most 1 time units; if unfinished, it returns to the back of the ready queue. Arrivals occurring exactly when the slice ends are entered before re-queuing the unfinished process.

Gantt chart (time interval : process):

```
0-1 : P1  |  1-2 : P2  |  2-3 : P3  |  3-4 : P4  |  4-5 : P5  |  5-6 : P1

6-7 : P3  |  7-8 : P5  |  8-9 : P1  |  9-10 : P5  |  10-11 : P1  |  11-12 : P5

12-13 : P1  |  13-14 : P5  |  14-15 : P1  |  15-16 : P1  |  16-17 : P1  |  17-18 : P1

18-19 : P1
```

Execution trace (every dispatch interval):

Interval	CPU / process	Duration	Remaining burst after interval
0–1	P1	1	9
1–2	P2	1	0
2–3	P3	1	1
3–4	P4	1	0

Interval	CPU / process	Duration	Remaining burst after interval
4–5	P5	1	4
5–6	P1	1	8
6–7	P3	1	0
7–8	P5	1	3
8–9	P1	1	7
9–10	P5	1	2
10–11	P1	1	6
11–12	P5	1	1
12–13	P1	1	5
13–14	P5	1	0
14–15	P1	1	4
15–16	P1	1	3
16–17	P1	1	2
17–18	P1	1	1
18–19	P1	1	0

Calculation formulas: CT = completion time; TAT = CT − AT; WT = TAT − BT; RT = first CPU start − AT.

Process-wise calculations:

PID	AT	BT	CT	TAT = CT−AT	WT = TAT−BT	RT
P1	0	10	19	19−0 = **19**	19−10 = **9**	0
P2	0	1	2	2−0 = **2**	2−1 = **1**	1
P3	0	2	7	7−0 = **7**	7−2 = **5**	2
P4	0	1	4	4−0 = **4**	4−1 = **3**	3
P5	0	5	14	14−0 = **14**	14−5 = **9**	4

Averages:

- Average WT = (9 + 1 + 5 + 3 + 9) / 5 = 27 / 5 = **5.4**.
- Average TAT = (19 + 2 + 7 + 4 + 14) / 5 = 46 / 5 = **9.2**.
- Average RT = (0 + 1 + 2 + 3 + 4) / 5 = 10 / 5 = **2**.
- Context switches (between different processes; do not count initial dispatch or final termination) = **14**.

Q9. For arrival times 0, 0.4, 1 and bursts 8, 4, 1, compare FCFS and SRTF.

Answer — Given process data:

Process	Arrival (AT)	Burst (BT)
P1	0	8
P2	0.4	4
P3	1	1

FCFS

Selection rule: Earliest arrival first, without preemption; use process-ID order for an equal-arrival tie.

Gantt chart (time interval : process):

```
0-8 : P1  |  8-12 : P2  |  12-13 : P3
```

Execution trace (every dispatch interval):

Interval	CPU / process	Duration	Remaining burst after interval
0–8	P1	8	0
8–12	P2	4	0
12–13	P3	1	0

Calculation formulas: CT = completion time; TAT = CT − AT; WT = TAT − BT; RT = first CPU start − AT.

Process-wise calculations:

PID	AT	BT	CT	TAT = CT−AT	WT = TAT−BT	RT
P1	0	8	8	8−0 = **8**	8−8 = **0**	0
P2	0.4	4	12	12−0.4 = **11.6**	11.6−4 = **7.6**	7.6
P3	1	1	13	13−1 = **12**	12−1 = **11**	11

Averages:

- Average WT = (0 + 7.6 + 11) / 3 = 18.6 / 3 = **6.2**.
- Average TAT = (8 + 11.6 + 12) / 3 = 31.6 / 3 = **10.5333**.
- Average RT = (0 + 7.6 + 11) / 3 = 18.6 / 3 = **6.2**.
- Context switches (between different processes; do not count initial dispatch or final termination) = **2**.

SJF (preemptive) / SRTF

Selection rule: At every arrival or completion, choose the available process with the shortest remaining time; retain the running process on an exact tie.

Gantt chart (time interval : process):

```
0-0.4 : P1  |  0.4-1 : P2  |  1-2 : P3  |  2-5.4 : P2  |  5.4-13 : P1
```

Execution trace (every dispatch interval):

Interval	CPU / process	Duration	Remaining burst after interval
0–0.4	P1	0.4	7.6
0.4–1	P2	0.6	3.4
1–2	P3	1	0
2–5.4	P2	3.4	0
5.4–13	P1	7.6	0

Calculation formulas: CT = completion time; TAT = CT − AT; WT = TAT − BT; RT = first CPU start − AT.

Process-wise calculations:

PID	AT	BT	CT	TAT = CT−AT	WT = TAT−BT	RT
P1	0	8	13	13−0 = **13**	13−8 = **5**	0
P2	0.4	4	5.4	5.4−0.4 = **5**	5−4 = **1**	0
P3	1	1	2	2−1 = **1**	1−1 = **0**	0

Averages:

- Average WT = (5 + 1 + 0) / 3 = 6 / 3 = **2**.
- Average TAT = (13 + 5 + 1) / 3 = 19 / 3 = **6.3333**.
- Average RT = (0 + 0 + 0) / 3 = 0 / 3 = **0**.
- Context switches (between different processes; do not count initial dispatch or final termination) = **4**.

Q10. Solve SRTF, preemptive Priority and both specified RR variants (q=2 actual arrivals; q=4 all arrivals at zero).

Answer — Given process data:

Process	Arrival (AT)	Burst (BT)	Priority
P1	0	11	2
P2	5	28	0
P3	12	2	3
P4	2	10	1
P5	9	16	4

SJF (preemptive) / SRTF

Selection rule: At every arrival or completion, choose the available process with the shortest remaining time; retain the running process on an exact tie.

Gantt chart (time interval : process):

`0-11 : P1 | 11-12 : P4 | 12-14 : P3 | 14-23 : P4 | 23-39 : P5 | 39-67 : P2`

Execution trace (every dispatch interval):

Interval	CPU / process	Duration	Remaining burst after interval
0–11	P1	11	0
11–12	P4	1	9
12–14	P3	2	0
14–23	P4	9	0
23–39	P5	16	0
39–67	P2	28	0

Calculation formulas: CT = completion time; TAT = CT − AT; WT = TAT − BT; RT = first CPU start − AT.

Process-wise calculations:

PID	AT	BT	CT	TAT = CT−AT	WT = TAT−BT	RT
P1	0	11	11	11−0 = **11**	11−11 = **0**	0
P2	5	28	67	67−5 = **62**	62−28 = **34**	34
P3	12	2	14	14−12 = **2**	2−2 = **0**	0
P4	2	10	23	23−2 = **21**	21−10 = **11**	9
P5	9	16	39	39−9 = **30**	30−16 = **14**	14

Averages:

- Average WT = (0 + 34 + 0 + 11 + 14) / 5 = 59 / 5 = **11.8**.
- Average TAT = (11 + 62 + 2 + 21 + 30) / 5 = 126 / 5 = **25.2**.
- Average RT = (0 + 34 + 0 + 9 + 14) / 5 = 57 / 5 = **11.4**.
- Context switches (between different processes; do not count initial dispatch or final termination) = **5**.

Priority (preemptive)

Selection rule: Choose the smallest numerical priority value; a strictly higher-priority arrival preempts. Equal-priority requests use FCFS order.

Gantt chart (time interval : process):

`0-2 : P1 | 2-5 : P4 | 5-33 : P2 | 33-40 : P4 | 40-49 : P1 | 49-51 : P3`

`51-67 : P5`

Execution trace (every dispatch interval):

Interval	CPU / process	Duration	Remaining burst after interval
0–2	P1	2	9
2–5	P4	3	7
5–33	P2	28	0
33–40	P4	7	0
40–49	P1	9	0
49–51	P3	2	0
51–67	P5	16	0

Calculation formulas: CT = completion time; TAT = CT − AT; WT = TAT − BT; RT = first CPU start − AT.

Process-wise calculations:

PID	AT	BT	CT	TAT = CT−AT	WT = TAT−BT	RT
P1	0	11	49	49−0 = **49**	49−11 = **38**	0
P2	5	28	33	33−5 = **28**	28−28 = **0**	0
P3	12	2	51	51−12 = **39**	39−2 = **37**	37
P4	2	10	40	40−2 = **38**	38−10 = **28**	0
P5	9	16	67	67−9 = **58**	58−16 = **42**	42

Averages:

- Average WT = (38 + 0 + 37 + 28 + 42) / 5 = 145 / 5 = **29**.
- Average TAT = (49 + 28 + 39 + 38 + 58) / 5 = 212 / 5 = **42.4**.
- Average RT = (0 + 0 + 37 + 0 + 42) / 5 = 79 / 5 = **15.8**.
- Context switches (between different processes; do not count initial dispatch or final termination) = **6**.

Round Robin, q = 2

Selection rule: Each ready process receives at most 2 time units; if unfinished, it returns to the back of the ready queue. Arrivals occurring exactly when the slice ends are entered before re-queuing the unfinished process.

Gantt chart (time interval : process):

```
0-2 : P1  |  2-4 : P4  |  4-6 : P1  |  6-8 : P4  |  8-10 : P2  |  10-12 : P1

12-14 : P4  |  14-16 : P5  |  16-18 : P2  |  18-20 : P3  |  20-22 : P1  |  22-24 : P4

24-26 : P5  |  26-28 : P2  |  28-30 : P1  |  30-32 : P4  |  32-34 : P5  |  34-36 : P2

36-37 : P1  |  37-39 : P5  |  39-41 : P2  |  41-43 : P5  |  43-45 : P2  |  45-47 : P5

47-49 : P2  |  49-51 : P5  |  51-53 : P2  |  53-55 : P5  |  55-57 : P2  |  57-59 : P2
```

```
59-61 : P2  |  61-63 : P2  |  63-65 : P2  |  65-67 : P2
```

Execution trace (every dispatch interval):

Interval	CPU / process	Duration	Remaining burst after interval
0–2	P1	2	9
2–4	P4	2	8
4–6	P1	2	7
6–8	P4	2	6
8–10	P2	2	26
10–12	P1	2	5
12–14	P4	2	4
14–16	P5	2	14
16–18	P2	2	24
18–20	P3	2	0
20–22	P1	2	3
22–24	P4	2	2
24–26	P5	2	12
26–28	P2	2	22
28–30	P1	2	1
30–32	P4	2	0
32–34	P5	2	10
34–36	P2	2	20
36–37	P1	1	0
37–39	P5	2	8
39–41	P2	2	18
41–43	P5	2	6
43–45	P2	2	16
45–47	P5	2	4
47–49	P2	2	14
49–51	P5	2	2
51–53	P2	2	12
53–55	P5	2	0
55–57	P2	2	10
57–59	P2	2	8
59–61	P2	2	6

Interval	CPU / process	Duration	Remaining burst after interval
61–63	P2	2	4
63–65	P2	2	2
65–67	P2	2	0

Calculation formulas: CT = completion time; TAT = CT − AT; WT = TAT − BT; RT = first CPU start − AT.

Process-wise calculations:

PID	AT	BT	CT	TAT = CT−AT	WT = TAT−BT	RT
P1	0	11	37	37−0 = **37**	37−11 = **26**	0
P2	5	28	67	67−5 = **62**	62−28 = **34**	3
P3	12	2	20	20−12 = **8**	8−2 = **6**	6
P4	2	10	32	32−2 = **30**	30−10 = **20**	0
P5	9	16	55	55−9 = **46**	46−16 = **30**	5

Averages:

- Average WT = (26 + 34 + 6 + 20 + 30) / 5 = 116 / 5 = **23.2**.
- Average TAT = (37 + 62 + 8 + 30 + 46) / 5 = 183 / 5 = **36.6**.
- Average RT = (0 + 3 + 6 + 0 + 5) / 5 = 14 / 5 = **2.8**.
- Context switches (between different processes; do not count initial dispatch or final termination) = **28**.

Alternate Round Robin question: q = 4, with all arrival times set to zero

RR variation (q=4, arrivals all zero)

Selection rule: Each ready process receives at most 4 time units; if unfinished, it returns to the back of the ready queue. Arrivals occurring exactly when the slice ends are entered before re-queuing the unfinished process.

Gantt chart (time interval : process):

```
0-4 : P1  |  4-8 : P2  |  8-10 : P3  |  10-14 : P4  |  14-18 : P5  |  18-22 : P1

22-26 : P2  |  26-30 : P4  |  30-34 : P5  |  34-37 : P1  |  37-41 : P2  |  41-43 : P4

43-47 : P5  |  47-51 : P2  |  51-55 : P5  |  55-59 : P2  |  59-63 : P2  |  63-67 : P2
```

Execution trace (every dispatch interval):

Interval	CPU / process	Duration	Remaining burst after interval
0–4	P1	4	7
4–8	P2	4	24
8–10	P3	2	0

Interval	CPU / process	Duration	Remaining burst after interval
10–14	P4	4	6
14–18	P5	4	12
18–22	P1	4	3
22–26	P2	4	20
26–30	P4	4	2
30–34	P5	4	8
34–37	P1	3	0
37–41	P2	4	16
41–43	P4	2	0
43–47	P5	4	4
47–51	P2	4	12
51–55	P5	4	0
55–59	P2	4	8
59–63	P2	4	4
63–67	P2	4	0

Calculation formulas: CT = completion time; TAT = CT − AT; WT = TAT − BT; RT = first CPU start − AT.

Process-wise calculations:

PID	AT	BT	CT	TAT = CT−AT	WT = TAT−BT	RT
P1	0	11	37	37−0 = **37**	37−11 = **26**	0
P2	0	28	67	67−0 = **67**	67−28 = **39**	4
P3	0	2	10	10−0 = **10**	10−2 = **8**	8
P4	0	10	43	43−0 = **43**	43−10 = **33**	10
P5	0	16	55	55−0 = **55**	55−16 = **39**	14

Averages:

- Average WT = (26 + 39 + 8 + 33 + 39) / 5 = 145 / 5 = **29**.
- Average TAT = (37 + 67 + 10 + 43 + 55) / 5 = 212 / 5 = **42.4**.
- Average RT = (0 + 4 + 8 + 10 + 14) / 5 = 36 / 5 = **7.2**.
- Context switches (between different processes; do not count initial dispatch or final termination) = **15**.

Important: This second version deliberately changes the arrivals for **RR only**. SRTF and Priority above retain their original given arrivals. Do not mix the two datasets in the same answer.

Q11. Schedule by Priority, using preemptive SJF only for equal priorities, and compute average TAT.

Answer — Given process data:

Process	Arrival (AT)	Burst (BT)	Priority
P1	7	5	1
P2	3	7	2
P3	10	8	1
P4	0	15	2
P5	20	3	1

Priority first, SRTF within equal priority

Selection rule: Smaller priority number wins; among processes of the same priority select the shortest remaining burst (preemptively).

Gantt chart (time interval : process):

`0-3 : P4 | 3-7 : P2 | 7-12 : P1 | 12-20 : P3 | 20-23 : P5 | 23-26 : P2`

`26-38 : P4`

Execution trace (every dispatch interval):

Interval	CPU / process	Duration	Remaining burst after interval
0–3	P4	3	12
3–7	P2	4	3
7–12	P1	5	0
12–20	P3	8	0
20–23	P5	3	0
23–26	P2	3	0
26–38	P4	12	0

Calculation formulas: CT = completion time; TAT = CT − AT; WT = TAT − BT; RT = first CPU start − AT.

Process-wise calculations:

PID	AT	BT	CT	TAT = CT−AT	WT = TAT−BT	RT
P1	7	5	12	12−7 = **5**	5−5 = **0**	0
P2	3	7	26	26−3 = **23**	23−7 = **16**	0
P3	10	8	20	20−10 = **10**	10−8 = **2**	2
P4	0	15	38	38−0 = **38**	38−15 = **23**	0
P5	20	3	23	23−20 = **3**	3−3 = **0**	0

Averages:

- Average WT = (0 + 16 + 2 + 23 + 0) / 5 = 41 / 5 = **8.2**.
- Average TAT = (5 + 23 + 10 + 38 + 3) / 5 = 79 / 5 = **15.8**.
- Average RT = (0 + 0 + 2 + 0 + 0) / 5 = 2 / 5 = **0.4**.
- Context switches (between different processes; do not count initial dispatch or final termination) = **6**.

Stepwise choices: At t=0 only P4 runs; at t=3 P2 (same priority, 7 remaining vs P4's 12) preempts; at t=7 P1 (higher priority 1) preempts; at t=10 P3 arrives with the same priority but P1 has only 2 remaining, so P1 completes; P3 runs 12–20; P5 (priority 1, burst 3) runs 20–23; the remaining priority-2 work runs P2 23–26 then P4 26–38. **Average turnaround time = 15.8.**

Q12. Solve SRTF, preemptive Priority, and RR (q=2) with average WT, TAT and RT.

Answer — Given process data:

Process	Arrival (AT)	Burst (BT)	Priority
P1	0	7	3
P2	3	2	2
P3	4	3	1
P4	4	1	1
P5	5	3	3

SJF (preemptive) / SRTF

Selection rule: At every arrival or completion, choose the available process with the shortest remaining time; retain the running process on an exact tie.

Gantt chart (time interval : process):

```
0-3 : P1  |  3-5 : P2  |  5-6 : P4  |  6-9 : P3  |  9-12 : P5  |  12-16 : P1
```

Execution trace (every dispatch interval):

Interval	CPU / process	Duration	Remaining burst after interval
0–3	P1	3	4
3–5	P2	2	0
5–6	P4	1	0
6–9	P3	3	0
9–12	P5	3	0
12–16	P1	4	0

Calculation formulas: CT = completion time; TAT = CT − AT; WT = TAT − BT; RT = first CPU start − AT.

Process-wise calculations:

PID	AT	BT	CT	TAT = CT−AT	WT = TAT−BT	RT
P1	0	7	16	16−0 = **16**	16−7 = **9**	0
P2	3	2	5	5−3 = **2**	2−2 = **0**	0
P3	4	3	9	9−4 = **5**	5−3 = **2**	2
P4	4	1	6	6−4 = **2**	2−1 = **1**	1
P5	5	3	12	12−5 = **7**	7−3 = **4**	4

Averages:

- Average WT = (9 + 0 + 2 + 1 + 4) / 5 = 16 / 5 = **3.2**.
- Average TAT = (16 + 2 + 5 + 2 + 7) / 5 = 32 / 5 = **6.4**.
- Average RT = (0 + 0 + 2 + 1 + 4) / 5 = 7 / 5 = **1.4**.
- Context switches (between different processes; do not count initial dispatch or final termination) = **5**.

Priority (preemptive)

Selection rule: Choose the smallest numerical priority value; a strictly higher-priority arrival preempts. Equal-priority requests use FCFS order.

Gantt chart (time interval : process):

```
0-3 : P1  |  3-4 : P2  |  4-7 : P3  |  7-8 : P4  |  8-9 : P2  |  9-13 : P1
```

```
13-16 : P5
```

Execution trace (every dispatch interval):

Interval	CPU / process	Duration	Remaining burst after interval
0–3	P1	3	4
3–4	P2	1	1
4–7	P3	3	0
7–8	P4	1	0
8–9	P2	1	0
9–13	P1	4	0
13–16	P5	3	0

Calculation formulas: CT = completion time; TAT = CT − AT; WT = TAT − BT; RT = first CPU start − AT.

Process-wise calculations:

PID	AT	BT	CT	TAT = CT−AT	WT = TAT−BT	RT
P1	0	7	13	13−0 = **13**	13−7 = **6**	0
P2	3	2	9	9−3 = **6**	6−2 = **4**	0
P3	4	3	7	7−4 = **3**	3−3 = **0**	0
P4	4	1	8	8−4 = **4**	4−1 = **3**	3
P5	5	3	16	16−5 = **11**	11−3 = **8**	8

Averages:

- Average WT = (6 + 4 + 0 + 3 + 8) / 5 = 21 / 5 = **4.2**.
- Average TAT = (13 + 6 + 3 + 4 + 11) / 5 = 37 / 5 = **7.4**.
- Average RT = (0 + 0 + 0 + 3 + 8) / 5 = 11 / 5 = **2.2**.
- Context switches (between different processes; do not count initial dispatch or final termination) = **6**.

Round Robin, q = 2

Selection rule: Each ready process receives at most 2 time units; if unfinished, it returns to the back of the ready queue. Arrivals occurring exactly when the slice ends are entered before re-queuing the unfinished process.

Gantt chart (time interval : process):

```
0-2 : P1  |  2-4 : P1  |  4-6 : P2  |  6-8 : P3  |  8-9 : P4  |  9-11 : P1
```

```
11-13 : P5  |  13-14 : P3  |  14-15 : P1  |  15-16 : P5
```

Execution trace (every dispatch interval):

Interval	CPU / process	Duration	Remaining burst after interval
0–2	P1	2	5
2–4	P1	2	3
4–6	P2	2	0
6–8	P3	2	1
8–9	P4	1	0
9–11	P1	2	1
11–13	P5	2	1
13–14	P3	1	0
14–15	P1	1	0
15–16	P5	1	0

Calculation formulas: CT = completion time; TAT = CT − AT; WT = TAT − BT; RT = first CPU start − AT.

Process-wise calculations:

PID	AT	BT	CT	TAT = CT−AT	WT = TAT−BT	RT
P1	0	7	15	15−0 = **15**	15−7 = **8**	0
P2	3	2	6	6−3 = **3**	3−2 = **1**	1
P3	4	3	14	14−4 = **10**	10−3 = **7**	2
P4	4	1	9	9−4 = **5**	5−1 = **4**	4
P5	5	3	16	16−5 = **11**	11−3 = **8**	6

Averages:

- Average WT = (8 + 1 + 7 + 4 + 8) / 5 = 28 / 5 = **5.6**.
- Average TAT = (15 + 3 + 10 + 5 + 11) / 5 = 44 / 5 = **8.8**.
- Average RT = (0 + 1 + 2 + 4 + 6) / 5 = 13 / 5 = **2.6**.
- Context switches (between different processes; do not count initial dispatch or final termination) = **8**.

Q13. Compare SJF and non-preemptive Priority for burst/priority data, assuming all processes arrive at zero.

Answer — Given process data:

Process	Arrival (AT)	Burst (BT)	Priority
P1	0	6	3
P2	0	8	1
P3	0	7	4
P4	0	3	2

The question supplies no arrival times, so **all arrivals are assumed t=0**; SJF and Priority are taken as **non-preemptive**.

SJF (non-preemptive)

Selection rule: At each CPU release, pick the available process with the smallest CPU burst and run it to completion.

Gantt chart (time interval : process):

```
0-3 : P4  |  3-9 : P1  |  9-16 : P3  |  16-24 : P2
```

Execution trace (every dispatch interval):

Interval	CPU / process	Duration	Remaining burst after interval
0–3	P4	3	0
3–9	P1	6	0

Interval	CPU / process	Duration	Remaining burst after interval
9–16	P3	7	0
16–24	P2	8	0

Calculation formulas: CT = completion time; TAT = CT − AT; WT = TAT − BT; RT = first CPU start − AT.

Process-wise calculations:

PID	AT	BT	CT	TAT = CT−AT	WT = TAT−BT	RT
P1	0	6	9	9−0 = **9**	9−6 = **3**	3
P2	0	8	24	24−0 = **24**	24−8 = **16**	16
P3	0	7	16	16−0 = **16**	16−7 = **9**	9
P4	0	3	3	3−0 = **3**	3−3 = **0**	0

Averages:

- Average WT = (3 + 16 + 9 + 0) / 4 = 28 / 4 = **7**.
- Average TAT = (9 + 24 + 16 + 3) / 4 = 52 / 4 = **13**.
- Average RT = (3 + 16 + 9 + 0) / 4 = 28 / 4 = **7**.
- Context switches (between different processes; do not count initial dispatch or final termination) = **3**.

Priority (non-preemptive)

Selection rule: Choose the smallest priority number; once selected, finish the burst. Equal-priority processes use FCFS.

Gantt chart (time interval : process):

```
0-8 : P2  |  8-11 : P4  |  11-17 : P1  |  17-24 : P3
```

Execution trace (every dispatch interval):

Interval	CPU / process	Duration	Remaining burst after interval
0–8	P2	8	0
8–11	P4	3	0
11–17	P1	6	0
17–24	P3	7	0

Calculation formulas: CT = completion time; TAT = CT − AT; WT = TAT − BT; RT = first CPU start − AT.

Process-wise calculations:

PID	AT	BT	CT	TAT = CT−AT	WT = TAT−BT	RT
P1	0	6	17	17−0 = **17**	17−6 = **11**	11
P2	0	8	8	8−0 = **8**	8−8 = **0**	0
P3	0	7	24	24−0 = **24**	24−7 = **17**	17
P4	0	3	11	11−0 = **11**	11−3 = **8**	8

Averages:

- Average WT = (11 + 0 + 17 + 8) / 4 = 36 / 4 = **9**.
- Average TAT = (17 + 8 + 24 + 11) / 4 = 60 / 4 = **15**.
- Average RT = (11 + 0 + 17 + 8) / 4 = 36 / 4 = **9**.
- Context switches (between different processes; do not count initial dispatch or final termination) = **3**.

Q14. Draw FCFS and preemptive SJF schedules and compute WT and TAT.

Answer — Given process data:

Process	Arrival (AT)	Burst (BT)
P1	0	2
P2	1	3
P3	3	1
P4	3	7

FCFS

Selection rule: Earliest arrival first, without preemption; use process-ID order for an equal-arrival tie.

Gantt chart (time interval : process):

```
0-2 : P1  |  2-5 : P2  |  5-6 : P3  |  6-13 : P4
```

Execution trace (every dispatch interval):

Interval	CPU / process	Duration	Remaining burst after interval
0–2	P1	2	0
2–5	P2	3	0
5–6	P3	1	0
6–13	P4	7	0

Calculation formulas: CT = completion time; TAT = CT − AT; WT = TAT − BT; RT = first CPU start − AT.

Process-wise calculations:

PID	AT	BT	CT	TAT = CT−AT	WT = TAT−BT	RT
P1	0	2	2	2−0 = **2**	2−2 = **0**	0
P2	1	3	5	5−1 = **4**	4−3 = **1**	1
P3	3	1	6	6−3 = **3**	3−1 = **2**	2
P4	3	7	13	13−3 = **10**	10−7 = **3**	3

Averages:

- Average WT = (0 + 1 + 2 + 3) / 4 = 6 / 4 = **1.5**.
- Average TAT = (2 + 4 + 3 + 10) / 4 = 19 / 4 = **4.75**.
- Average RT = (0 + 1 + 2 + 3) / 4 = 6 / 4 = **1.5**.
- Context switches (between different processes; do not count initial dispatch or final termination) = **3**.

SJF (preemptive) / SRTF

Selection rule: At every arrival or completion, choose the available process with the shortest remaining time; retain the running process on an exact tie.

Gantt chart (time interval : process):

```
0-2 : P1  |  2-3 : P2  |  3-4 : P3  |  4-6 : P2  |  6-13 : P4
```

Execution trace (every dispatch interval):

Interval	CPU / process	Duration	Remaining burst after interval
0–2	P1	2	0
2–3	P2	1	2
3–4	P3	1	0
4–6	P2	2	0
6–13	P4	7	0

Calculation formulas: CT = completion time; TAT = CT − AT; WT = TAT − BT; RT = first CPU start − AT.

Process-wise calculations:

PID	AT	BT	CT	TAT = CT−AT	WT = TAT−BT	RT
P1	0	2	2	2−0 = **2**	2−2 = **0**	0
P2	1	3	6	6−1 = **5**	5−3 = **2**	1
P3	3	1	4	4−3 = **1**	1−1 = **0**	0
P4	3	7	13	13−3 = **10**	10−7 = **3**	3

Averages:

- Average WT = (0 + 2 + 0 + 3) / 4 = 5 / 4 = **1.25**.
- Average TAT = (2 + 5 + 1 + 10) / 4 = 18 / 4 = **4.5**.
- Average RT = (0 + 1 + 0 + 3) / 4 = 4 / 4 = **1**.
- Context switches (between different processes; do not count initial dispatch or final termination) = **4**.

Q15. Draw Round Robin (q=4) for bursts 24, 3, 3, with all arrivals at zero.

Answer — Given process data:

Process	Arrival (AT)	Burst (BT)
P1	0	24
P2	0	3
P3	0	3

Round Robin, q = 4

Selection rule: Each ready process receives at most 4 time units; if unfinished, it returns to the back of the ready queue. Arrivals occurring exactly when the slice ends are entered before re-queuing the unfinished process.

Gantt chart (time interval : process):

```
0-4 : P1  |  4-7 : P2  |  7-10 : P3  |  10-14 : P1  |  14-18 : P1  |  18-22 : P1
22-26 : P1  |  26-30 : P1
```

Execution trace (every dispatch interval):

Interval	CPU / process	Duration	Remaining burst after interval
0–4	P1	4	20
4–7	P2	3	0
7–10	P3	3	0
10–14	P1	4	16
14–18	P1	4	12
18–22	P1	4	8
22–26	P1	4	4
26–30	P1	4	0

Calculation formulas: CT = completion time; TAT = CT − AT; WT = TAT − BT; RT = first CPU start − AT.

Process-wise calculations:

PID	AT	BT	CT	TAT = CT−AT	WT = TAT−BT	RT
P1	0	24	30	30−0 = **30**	30−24 = **6**	0
P2	0	3	7	7−0 = **7**	7−3 = **4**	4
P3	0	3	10	10−0 = **10**	10−3 = **7**	7

Averages:

- Average WT = (6 + 4 + 7) / 3 = 17 / 3 = **5.6667**.
- Average TAT = (30 + 7 + 10) / 3 = 47 / 3 = **15.6667**.
- Average RT = (0 + 4 + 7) / 3 = 11 / 3 = **3.6667**.
- Context switches (between different processes; do not count initial dispatch or final termination) = **3**.

Check: RR timeline: P1 gets 0–4 (20 left), P2 4–7 (finishes), P3 7–10 (finishes), then P1 has no competitor and continues for five more 4-unit slices: 10–14, 14–18, 18–22, 22–26, 26–30. Thus P1 completes at **30 ms**. (Unit 2, p. 46, uses this exact case.)

Q16. Solve SRTF for P1(0,20), P2(15,25), P3(30,10), P4(45,15).

Answer — Given process data:

Process	Arrival (AT)	Burst (BT)
P1	0	20
P2	15	25
P3	30	10
P4	45	15

SJF (preemptive) / SRTF

Selection rule: At every arrival or completion, choose the available process with the shortest remaining time; retain the running process on an exact tie.

Gantt chart (time interval : process):

```
0-20 : P1  |  20-30 : P2  |  30-40 : P3  |  40-55 : P2  |  55-70 : P4
```

Execution trace (every dispatch interval):

Interval	CPU / process	Duration	Remaining burst after interval
0–20	P1	20	0
20–30	P2	10	15
30–40	P3	10	0
40–55	P2	15	0
55–70	P4	15	0

Calculation formulas: CT = completion time; TAT = CT − AT; WT = TAT − BT; RT = first CPU start − AT.

Process-wise calculations:

PID	AT	BT	CT	TAT = CT−AT	WT = TAT−BT	RT
P1	0	20	20	20−0 = **20**	20−20 = **0**	0
P2	15	25	55	55−15 = **40**	40−25 = **15**	5
P3	30	10	40	40−30 = **10**	10−10 = **0**	0
P4	45	15	70	70−45 = **25**	25−15 = **10**	10

Averages:

- Average WT = (0 + 15 + 0 + 10) / 4 = 25 / 4 = **6.25**.
- Average TAT = (20 + 40 + 10 + 25) / 4 = 95 / 4 = **23.75**.
- Average RT = (0 + 5 + 0 + 10) / 4 = 15 / 4 = **3.75**.
- Context switches (between different processes; do not count initial dispatch or final termination) = **4**.

Q17. Find completion time of A for RR (q=1) with A=4, B=1, C=8, D=1.

Answer — Given process data:

Process	Arrival (AT)	Burst (BT)
A	0	4
B	0	1
C	0	8
D	0	1

Round Robin, q = 1

Selection rule: Each ready process receives at most 1 time units; if unfinished, it returns to the back of the ready queue. Arrivals occurring exactly when the slice ends are entered before re-queuing the unfinished process.

Gantt chart (time interval : process):

```
0-1 : A  |  1-2 : B  |  2-3 : C  |  3-4 : D  |  4-5 : A  |  5-6 : C
```

```
6-7 : A  |  7-8 : C  |  8-9 : A  |  9-10 : C  |  10-11 : C  |  11-12 : C
```

```
12-13 : C  |  13-14 : C
```

Execution trace (every dispatch interval):

Interval	CPU / process	Duration	Remaining burst after interval
0–1	A	1	3

Interval	CPU / process	Duration	Remaining burst after interval
1–2	B	1	0
2–3	C	1	7
3–4	D	1	0
4–5	A	1	2
5–6	C	1	6
6–7	A	1	1
7–8	C	1	5
8–9	A	1	0
9–10	C	1	4
10–11	C	1	3
11–12	C	1	2
12–13	C	1	1
13–14	C	1	0

Calculation formulas: CT = completion time; TAT = CT − AT; WT = TAT − BT; RT = first CPU start − AT.

Process-wise calculations:

PID	AT	BT	CT	TAT = CT−AT	WT = TAT−BT	RT
A	0	4	9	9−0 = **9**	9−4 = **5**	0
B	0	1	2	2−0 = **2**	2−1 = **1**	1
C	0	8	14	14−0 = **14**	14−8 = **6**	2
D	0	1	4	4−0 = **4**	4−1 = **3**	3

Averages:

- Average WT = (5 + 1 + 6 + 3) / 4 = 15 / 4 = **3.75**.
- Average TAT = (9 + 2 + 14 + 4) / 4 = 29 / 4 = **7.25**.
- Average RT = (0 + 1 + 2 + 3) / 4 = 6 / 4 = **1.5**.
- Context switches (between different processes; do not count initial dispatch or final termination) = **9**.

Required final answer: A finishes its fourth (last) quantum at **time 9**; therefore **completion time of A = 9**.

Q18. Find the number of SRTF context switches when bursts are 10,20,30 and arrivals 0,2,6.

Answer — Given process data:

Process	Arrival (AT)	Burst (BT)
P1	0	10
P2	2	20
P3	6	30

SJF (preemptive) / SRTF

Selection rule: At every arrival or completion, choose the available process with the shortest remaining time; retain the running process on an exact tie.

Gantt chart (time interval : process):

```
0-10 : P1  |  10-30 : P2  |  30-60 : P3
```

Execution trace (every dispatch interval):

Interval	CPU / process	Duration	Remaining burst after interval
0–10	P1	10	0
10–30	P2	20	0
30–60	P3	30	0

Calculation formulas: CT = completion time; TAT = CT − AT; WT = TAT − BT; RT = first CPU start − AT.

Process-wise calculations:

PID	AT	BT	CT	TAT = CT−AT	WT = TAT−BT	RT
P1	0	10	10	10−0 = **10**	10−10 = **0**	0
P2	2	20	30	30−2 = **28**	28−20 = **8**	8
P3	6	30	60	60−6 = **54**	54−30 = **24**	24

Averages:

- Average WT = (0 + 8 + 24) / 3 = 32 / 3 = **10.6667**.
- Average TAT = (10 + 28 + 54) / 3 = 92 / 3 = **30.6667**.
- Average RT = (0 + 8 + 24) / 3 = 32 / 3 = **10.6667**.
- Context switches (between different processes; do not count initial dispatch or final termination) = **2**.

Why no preemptions at t=2 or t=6? At t=2, P1 remaining 8 < P2 burst 20. At t=6, P1 remaining 4 < P3 burst 30. P1 finishes at t=10; then P2 runs to t=30; then P3 to t=60. There are process changes **P1→P2** at t=10 and **P2→P3** at t=30: **2 context switches**.

Q19. Determine long-run CPU utilization under FCFS and RR (q=10) for 3 processes with repeated CPU=20 and I/O=80.

Answer — Given process data:

Each of P1, P2 and P3 repeatedly executes a **20 ms CPU burst**, then performs **80 ms I/O**, with I/O overlapping across processes. All three initially arrive around t=0. Assume zero context-switch overhead and that I/O begins immediately after each CPU burst. We must inspect the repeating pattern, not merely the initial busy interval.

(a) FCFS

- P1 executes CPU from **0–20**, then I/O **20–100**.
- P2 executes CPU from **20–40**, then I/O **40–120**.
- P3 executes CPU from **40–60**, then I/O **60–140**.
- No process is ready from **60–100**, so the CPU is idle.
- At 100, 120, 140 the same pattern repeats: busy **100–160**, idle **160–200**.

One steady-state cycle is **100 ms**, containing **60 ms** useful CPU work and **40 ms** idle time.

CPU utilization (FCFS) = (60/100) × 100 = 60%.

(b) RR, quantum 10 ms

- First round: P1 **0–10**, P2 **10–20**, P3 **20–30**.
- Second round: P1 **30–40** (now I/O 40–120), P2 **40–50** (I/O 50–130), P3 **50–60** (I/O 60–140).
- All three are in I/O from **60–120**, so CPU is idle for 60 ms.
- New cycle: P1 **120–130**, P2 **130–140**, P3 **140–150**, P1 **150–160**, P2 **160–170**, P3 **170–180**, then idle **180–240**.

The repeating cycle is **120 ms** with **60 ms** CPU execution and **60 ms** idle time.

CPU utilization (RR) = (60/120) × 100 = 50%.

Conclusion: FCFS = **60%**, RR (q=10) = **50%**. Here RR causes the I/O periods to overlap more closely, increasing idle time. This conclusion assumes 20 ms is each process's full CPU burst before an 80 ms I/O, as stated.

Section 4 — Real-Time CPU Scheduling (3 questions)

Q1. Explain the distinguishing characteristics of a real-time operating system (RTOS).

Answer:

A **real-time operating system (RTOS)** must provide **predictable, bounded response times**, especially when tasks have deadlines. Unlike general-purpose OS scheduling, correctness depends on **when** a result is produced as well as **what** result is produced.

Features:

1. **Deadline awareness:** tasks have release times, execution demands and deadlines.
2. **Deterministic scheduling:** bounded interrupt and dispatch latencies are essential (Unit 2, pp. 53–54).

3. **Priority-aware preemption:** urgent work should be able to run rapidly.
4. **Predictable synchronization:** shared-resource access must limit blocking and priority inversion.
5. **Reliability:** system must behave consistently under expected load.

Types: A **hard real-time system** must meet a deadline; a late result may be useless (e.g., airbag control). A **soft real-time system** prefers timeliness but may tolerate occasional deadline misses (e.g., streamed media). **Event latency** is the time from an event to service; it includes interrupt latency and dispatch delay. Lower, bounded latency is especially important for hard real time.

Q2. Compare Rate Monotonic Scheduling (RMS) and Earliest Deadline First (EDF).

Answer:

Note: The supplied Unit-2 PDF covers RTOS types and latencies (pp. 53–54), but does not detail RMS/EDF; the following is standard additional scheduling theory needed to answer the question.

Property	Rate Monotonic Scheduling (RMS)	Earliest Deadline First (EDF)
Priority	Fixed, assigned by period	Dynamic, assigned by nearest absolute deadline
Higher priority	Task with shorter period	Job whose deadline occurs sooner
Scheduling	Usually preemptive	Usually preemptive
Standard assumption	Independent periodic tasks with D = period	Independent preemptible jobs, one CPU, D = period for basic utilization test
Feasibility benchmark	Sufficient Liu–Layland bound: U ≤ n(2^(1/n) − 1)	If total utilization U ≤ 1, schedulable under ideal uniprocessor assumptions
Benefit	Simple priorities; predictable	More processor utilization is schedulable
Limitation	Some feasible task sets fail with fixed priorities	Dynamic priority/deadline management adds complexity

For 3 periodic tasks the RMS sufficient bound is `3(2^(1/3) − 1) ≈ 0.7798 = 77.98%`. The bound is sufficient, **not necessary**, so a task set above it may still meet deadlines under RMS.

Q3. For A(Period 4, C=1, D=4), B(6,2,6) and C(8,2,8), calculate utilization and EDF schedule over t=0 to 12.

Answer:

Given: A has `T_A=4`, `C_A=1`, `D_A=4`; B has `T_B=6`, `C_B=2`, `D_B=6`; C has `T_C=8`, `C_C=2`, `D_C=8`. Each deadline is measured from its job's release.

Step 1: Utilization

`U = C_A/T_A + C_B/T_B + C_C/T_C`

`= 1/4 + 2/6 + 2/8 = 3/12 + 4/12 + 3/12 = 10/12 = 5/6`

Therefore **U = 0.8333 = 83.33%**, below the ideal EDF limit of 100%.

Step 2: Release times and absolute deadlines

Job	Release	Execution	Absolute deadline
A1	0	1	4
A2	4	1	8
A3	8	1	12
B1	0	2	6
B2	6	2	12
C1	0	2	8
C2	8	2	16

Step 3: EDF rule: at each decision point run the ready job with the smallest **absolute** deadline; if deadlines tie, retain the earlier released/previously running job.

Gantt chart: `0–1 A1 | 1–3 B1 | 3–5 C1 | 5–6 A2 | 6–8 B2 | 8–9 A3 | 9–11 C2 | 11–12 IDLE`.

Step 4: Deadline verification

Job	Completion	Deadline	Meets deadline?
A1	1	4	Yes
B1	3	6	Yes
C1	5	8	Yes
A2	6	8	Yes
B2	8	12	Yes
A3	9	12	Yes
C2	11	16	Yes

During the first 12 units total useful work is 3(A)+4(B)+4(C)=11 units, with one idle unit; this **11/12 observed-window busy ratio** differs from the long-term **5/6 periodic utilization** because C2 executes before t=12 while its deadline is later. **Final answer: long-term utilization 83.33%; all shown EDF jobs meet their deadlines.**

Section 5 — Process Synchronization: Background (4 questions)

Q1. Define race condition and mention methods to avoid it.

Answer:

A **race condition** occurs when two or more processes/threads concurrently access and manipulate shared data, and the outcome depends on the order in which the accesses are interleaved (Unit 3, p. 7).

Prevention techniques: (1) protect shared updates with a **mutex/lock**, (2) use binary/counting **semaphores**, (3) use **monitors** or condition variables, (4) use atomic operations such as **test_and_set** or **compare_and_swap**, (5) design correct critical sections satisfying mutual exclusion, progress and bounded waiting, or (6) use message passing to avoid unsafe shared-memory access. **Principle:** updates that must happen together must be atomic or correctly synchronized.

Q2. What is a race condition? Explain with a suitable example.

Answer:

Suppose `counter=5`. A producer increments it and a consumer decrements it; logically the final value should remain **5**.

Machine-level steps:

```
Producer: R1 = counter; R1 = R1 + 1; counter = R1;
Consumer: R2 = counter; R2 = R2 - 1; counter = R2;
```

Unsafe interleaving:

Step	Instruction	R1	R2	counter
Start	Initial	—	—	5
1	Producer R1 = counter	5	—	5
2	Producer R1++	6	—	5
3	Consumer R2 = counter	6	5	5
4	Consumer R2--	6	4	5
5	Producer counter = R1	6	4	6
6	Consumer counter = R2	6	4	**4**

The final answer is **4**, not 5: the increment was lost. If the final two stores reverse, it can become **6**. A mutex around each update prevents this race (Unit 3, pp. 5–7).

Q3. Illustrate race condition with an example.

Answer:

Example — two threads updating a bank account: balance = 100; Thread T1 deposits 50; Thread T2 withdraws 30. Correct final balance = `100+50−30=120`.

1. T1 reads `100`, computes `150`, but has not stored it.
2. T2 also reads `100`, computes `70` and stores `70`.
3. T1 stores its old computed result `150`.

4. Final balance is **150** instead of 120; T2's withdrawal is lost.

If T2 stores last, final could instead be **70**. **Reason:** read–modify–write is not an atomic sequence. **Solution:** enclose each entire balance update in a mutex-protected critical section, ensuring one updater completes before the other begins.

Q4. Two concurrent functions P1 and P2 use B=2: P1: C=B−1; B=2C. P2: D=2B; B=D−1. How many distinct final values of B are possible?

Answer:

Treat each assignment as an atomic step but allow interleaving, while preserving **P1's internal order** and **P2's internal order**.

Let `a: C=B−1`, `b: B=2C`, `c: D=2B`, `d: B=D−1`. The legal instruction orders are exactly `abcd`, `acbd`, `acdb`, `cabd`, `cadb`, `cdab`.

Trace every legal order, starting B=2:

Order	Intermediate operations	Final B
a b c d	C=1; B=2; D=4; B=3	**3**
a c b d	C=1; D=4; B=2; B=3	**3**
a c d b	C=1; D=4; B=3; B=2	**2**
c a b d	D=4; C=1; B=2; B=3	**3**
c a d b	D=4; C=1; B=3; B=2	**2**
c d a b	D=4; B=3; C=2; B=4	**4**

Distinct final values = {2, 3, 4}. Total = 3 distinct values. This demonstrates why concurrent processes require synchronization.

Section 6 — Critical Section Problem (5 questions)

Q1. Define the critical-section problem, show its general structure and explain its requirements.

Answer:

A **critical section (CS)** is a part of a process that reads/writes shared variables, shared memory, files or other common resources. The **critical-section problem** is to design an entry/exit protocol so concurrent processes access these resources safely (Unit 3, pp. 8–10).

General structure of process Pi:

```
while (1) {
    // Entry section: request permission
    // Critical section: use/update shared resource
    // Exit section: release permission
    // Remainder section: unrelated work
}
```

Three compulsory properties:

1. **Mutual exclusion:** if Pi is inside its CS, no other process can simultaneously execute a CS for the same shared resource.
2. **Progress:** when no process is inside the CS and interested processes exist, the choice of the next entrant is not postponed indefinitely; unrelated remainder-section processes must not prevent a decision.
3. **Bounded waiting:** after Pi requests entry, there is a finite bound on how often other processes may enter before Pi gets its turn; prevents starvation.

A correct solution should not assume fixed CPU speeds or a favorable timing order.

Q2. Define the critical-section problem with an example and explain any two conditions.

Answer:

Definition: When multiple concurrent processes update the same shared data, the OS must control entry so the shared state remains consistent.

Example: `balance=100`. P1 executes `balance=balance+50` and P2 executes `balance=balance−30`. Their read–modify–write sequences can interleave and produce the wrong answer. Encapsulating each update in a CS prevents the race.

Condition 1 — Mutual exclusion: while P1 changes the balance, P2 cannot access the same update CS. Hence each transaction observes a consistent balance.

Condition 2 — Progress: if no transaction is updating the balance, then a waiting transaction must eventually be selected; a process doing unrelated work cannot hold up entry. A complete solution additionally requires **bounded waiting** so neither P1 nor P2 starves.

Q3. Illustrate the critical-section problem and list the necessary conditions.

Answer:

Diagram:

```
Process P1                          Process P2
Entry: acquire(lock)                Entry: acquire(lock)
       |                                   |
       v                                   v
Critical: shared++;                 Critical: shared--;
       |                                   |
Exit: release(lock)                 Exit: release(lock)
       |                                   |
Remainder work                      Remainder work
```

At any instant **at most one** process is in the protected critical region. **Required conditions:** (i) mutual exclusion, (ii) progress, and (iii) bounded waiting (Unit 3, p. 10). **Mutual exclusion** keeps writes from colliding, **progress** guarantees selection when the resource is free, and **bounded waiting** restricts how long an interested process can be bypassed.

Q4. Why is mutual exclusion important in solving the critical-section problem?

Answer:

Mutual exclusion ensures that shared-data modifications behave as though they occur sequentially, not as unsafe interleavings.

Consider `x=0`, P1 executes `x=x+1`, and P2 does the same. Without mutual exclusion both can read 0 and store 1, leaving the incorrect result **1**. With a lock, one completes `0→1`; the other then completes `1→2`, giving **2**.

Importance: (1) prevents lost updates, (2) preserves integrity of shared tables/files, (3) avoids race conditions, and (4) supports correct concurrent execution. **Limitation:** mutual exclusion alone does not guarantee progress or bounded waiting; those must also be examined.

Q5. Analyze S1/S2 Boolean protocol for initial (0,0) and (0,1); are mutual exclusion and progress guaranteed?

Answer:

Given:

```c
// P1
while (S1 == S2) ;
/* critical section */
S1 = S2;

// P2
while (S1 != S2) ;
/* critical section */
S2 = !S1;
```

Case 1: S1=0, S2=0:

1. P1 checks `S1==S2` → `0==0`, **true**, so P1 busy-waits.
2. P2 checks `S1!=S2` → `0!=0`, **false**, so P2 enters its CS.

3. On P2 exit, `S2=!S1=1`; now `(S1,S2)=(0,1)`.
4. P1's condition `0==1` is false; P1 may enter its CS. On exit, `S1=S2=1`, returning the turn to P2.

Case 2: S1=0, S2=1:

1. P1 checks `0==1` → false, so **P1 enters**.
2. P2 checks `0!=1` → true, so **P2 waits**.
3. On P1 exit, `S1=S2=1`; now P2 can enter and later set `S2=!S1=0`.

Mutual exclusion: Yes in both cases, under atomic reads/writes: the two while-conditions are complementary; both cannot be false simultaneously for a fixed pair of shared values, and the variables change only on exit.

Progress: No in either case. If `(S1,S2)=(0,0)` and P2 remains forever in its remainder section, P1 waits even though the CS is empty. Similarly if `(0,1)` and P1 is not interested, P2 waits. This is **strict alternation**, which violates progress despite preserving mutual exclusion. **Conclusion:** not a fully valid CS solution.

Section 7 — Peterson's Solution (5 questions)

Q1. Explain Peterson's solution to the critical-section problem.

Answer:

Peterson's algorithm for two processes Pi and Pj, with `j = 1–i`:

```
// Shared variables
bool flag[2] = {false, false};
int turn;

// Process Pi, where j = 1-i
while (true) {
    flag[i] = true;             // declare interest
    turn = j;                   // allow j to win a simultaneous tie
    while (flag[j] && turn == j)
        ;                       // busy wait

    /* CRITICAL SECTION */

    flag[i] = false;            // leave critical section
    /* REMAINDER SECTION */
}
```

Meaning: `flag[i]` says Pi wants to enter; `turn` resolves competition if both want in. If only Pi wants entry, `flag[j]=false`, so it enters immediately. If both want entry, the *last* writer to `turn` yields, so one enters and the other waits. (Unit 3, pp. 13–15.)

Result: Peterson's design uses only the shared `flag[2]` and `turn` variables and satisfies the three classical requirements in its ideal memory model.

Correctness proof (assuming sequentially consistent atomic reads/writes):

1. **Mutual exclusion:** Assume both Pi and Pj are in their CS. Both flags must be true. For Pi to pass its while-loop, `turn ≠ j`, i.e., `turn=i`. For Pj to pass, `turn ≠ i`, i.e., `turn=j`. Since `turn` cannot equal `i` and `j` simultaneously, this is impossible. Thus only one enters.
2. **Progress:** If only Pi is interested, `flag[j]=false` and Pi enters. If both are interested, the final value of `turn` makes exactly one while-condition false, selecting one process; an uninterested process cannot prevent entry.
3. **Bounded waiting:** After Pi sets `flag[i]=true`, Pj can enter at most once before Pi. On finishing Pj sets `flag[j]=false`; if Pj immediately retries, it writes `turn=i`, giving Pi priority. Hence Pi cannot be bypassed indefinitely.

Restrictions: only two processes, spinning wastes CPU cycles, shared-memory ordering must be respected (compiler/CPU reordering may break naïve code on modern hardware), and it is not a drop-in replacement for production mutexes.

Q2. Write Peterson's two-process synchronization algorithm and explain each statement.

Answer:

Peterson's algorithm for two processes Pi and Pj, with `j = 1–i`:

```
// Shared variables
bool flag[2] = {false, false};
int turn;

// Process Pi, where j = 1-i
while (true) {
    flag[i] = true;             // declare interest
    turn = j;                   // allow j to win a simultaneous tie
    while (flag[j] && turn == j)
        ;                       // busy wait

    /* CRITICAL SECTION */

    flag[i] = false;            // leave critical section
    /* REMAINDER SECTION */
}
```

Meaning: `flag[i]` says Pi wants to enter; `turn` resolves competition if both want in. If only Pi wants entry, `flag[j]=false`, so it enters immediately. If both want entry, the *last* writer to `turn` yields, so one enters and the other waits. (Unit 3, pp. 13–15.)

Statement-by-statement explanation: `flag[i]=true` announces Pi's entry request; `turn=j` politely gives the other contender the tie break; `while(flag[j] && turn==j)` delays Pi only when Pj is also interested and has been favored; the CS modifies shared data; `flag[i]=false` tells Pj that Pi has departed. **Example:** if both flag values are true and the final `turn` value is `1`, P0 waits and P1 enters; after P1 clears `flag[1]`, P0 proceeds.

Q3. State the requirements of a critical section and illustrate Peterson's solution.

Answer:

Requirements: mutual exclusion, **progress** and **bounded waiting** (Unit 3, p. 10). Mutual exclusion forbids simultaneous access; progress requires a timely choice when CS is empty; bounded waiting prevents indefinite overtaking.

Peterson's algorithm for two processes Pi and Pj, with `j = 1–i`:

```
// Shared variables
bool flag[2] = {false, false};
int turn;

// Process Pi, where j = 1-i
while (true) {
    flag[i] = true;             // declare interest
    turn = j;                  // allow j to win a simultaneous tie
    while (flag[j] && turn == j)
        ;                      // busy wait

    /* CRITICAL SECTION */

    flag[i] = false;            // leave critical section
    /* REMAINDER SECTION */
}
```

Meaning: `flag[i]` says Pi wants to enter; `turn` resolves competition if both want in. If only Pi wants entry, `flag[j]=false`, so it enters immediately. If both want entry, the *last* writer to `turn` yields, so one enters and the other waits. (Unit 3, pp. 13–15.)

Illustration: Initially `flag[0]=flag[1]=false`. P0 sets `flag[0]=true`, `turn=1`. If P1 does not request access, P0 enters immediately. If P1 requests too, it sets `flag[1]=true`, `turn=0`; now P0's wait is false (`turn ≠ 1`) and P0 enters while P1 waits. On leaving, P0 clears its flag and P1 enters. Thus the variables control entry correctly.

Q4. Prove Peterson's solution satisfies all requirements.

Answer:

Peterson's algorithm for two processes Pi and Pj, with `j = 1–i`:

```
// Shared variables
bool flag[2] = {false, false};
int turn;

// Process Pi, where j = 1-i
while (true) {
    flag[i] = true;             // declare interest
    turn = j;                   // allow j to win a simultaneous tie
    while (flag[j] && turn == j)
        ;                       // busy wait

    /* CRITICAL SECTION */

    flag[i] = false;            // leave critical section
    /* REMAINDER SECTION */
}
```

Meaning: `flag[i]` says Pi wants to enter; `turn` resolves competition if both want in. If only Pi wants entry, `flag[j]=false`, so it enters immediately. If both want entry, the *last* writer to `turn` yields, so one enters and the other waits. (Unit 3, pp. 13–15.)

Correctness proof (assuming sequentially consistent atomic reads/writes):

1. **Mutual exclusion:** Assume both Pi and Pj are in their CS. Both flags must be true. For Pi to pass its while-loop, `turn ≠ j`, i.e., `turn=i`. For Pj to pass, `turn ≠ i`, i.e., `turn=j`. Since `turn` cannot equal `i` and `j` simultaneously, this is impossible. Thus only one enters.
2. **Progress:** If only Pi is interested, `flag[j]=false` and Pi enters. If both are interested, the final value of `turn` makes exactly one while-condition false, selecting one process; an uninterested process cannot prevent entry.
3. **Bounded waiting:** After Pi sets `flag[i]=true`, Pj can enter at most once before Pi. On finishing Pj sets `flag[j]=false`; if Pj immediately retries, it writes `turn=i`, giving Pi priority. Hence Pi cannot be bypassed indefinitely.

Restrictions: only two processes, spinning wastes CPU cycles, shared-memory ordering must be respected (compiler/CPU reordering may break naïve code on modern hardware), and it is not a drop-in replacement for production mutexes.

Q5. Explain the structure of Pi in Peterson's solution and list disadvantages.

Answer:

Peterson's algorithm for two processes Pi and Pj, with `j = 1–i`:

```
// Shared variables
bool flag[2] = {false, false};
int turn;

// Process Pi, where j = 1-i
while (true) {
    flag[i] = true;             // declare interest
    turn = j;                   // allow j to win a simultaneous tie
    while (flag[j] && turn == j)
        ;                       // busy wait

    /* CRITICAL SECTION */

    flag[i] = false;            // leave critical section
    /* REMAINDER SECTION */
}
```

Meaning: `flag[i]` says Pi wants to enter; `turn` resolves competition if both want in. If only Pi wants entry, `flag[j]=false`, so it enters immediately. If both want entry, the *last* writer to `turn` yields, so one enters and the other waits. (Unit 3, pp. 13–15.)

Process flow: request (flag), resolve contention (turn), wait if appropriate, enter CS, clear flag on exit, execute remainder section, repeat. **Advantages:** simple, no special hardware primitive, illustrates all three CS requirements in the ideal model. **Disadvantages:** (1) supports only **two** processes as written; (2) **busy waiting** consumes CPU time; (3) correct behavior needs strong visibility/ordering of reads and writes, not automatically ensured on modern multiprocessors; (4) atomic stores alone are insufficient if memory operations are reordered; (5) production code should use suitable OS/hardware-supported synchronization instead. Unit 3, p. 16, specifically notes limitations of purely software solutions on modern architectures.

Section 8 — Synchronization Hardware (8 questions)

Q1. Explain a hardware solution to the critical-section problem for two processes.

Answer:

Hardware provides **atomic read–modify–write operations**, so no competing CPU can observe a partially completed lock acquisition. A two-process (and multiprocess) solution may use **test_and_set**.

Atomic `test_and_set` primitive:

```
bool test_and_set(bool *target) { // executed ATOMICALLY by hardware
    bool old = *target;
    *target = true;
    return old;
}
```

Spinlock using it:

```
bool lock = false;      // shared, initially unlocked
while (true) {
    while (test_and_set(&lock))
        ;               // busy wait
    /* CRITICAL SECTION */
    lock = false;       // atomic unlock/store
    /* REMAINDER SECTION */
}
```

If a process finds `lock=false`, it atomically receives `false` and sets `lock=true`; it enters. All other calls return `true` and spin. The owner releases by storing `false`.

Why it works: the lock can change from false to true for only one process per successful atomic acquisition, preserving **mutual exclusion**. However, this simple TAS spinlock does **not** guarantee bounded waiting: one process can repeatedly win the lock. A scheduling assumption that the lock holder eventually runs and releases it is also needed for progress. (Unit 3, pp. 16–18.)

Q2. Illustrate test_and_set and compare_and_swap synchronization primitives.

Answer:

Atomic `test_and_set` primitive:

```
bool test_and_set(bool *target) { // executed ATOMICALLY by hardware
    bool old = *target;
    *target = true;
    return old;
}
```

Spinlock using it:

```
bool lock = false;      // shared, initially unlocked
while (true) {
    while (test_and_set(&lock))
        ;               // busy wait
    /* CRITICAL SECTION */
    lock = false;       // atomic unlock/store
    /* REMAINDER SECTION */
}
```

If a process finds `lock=false`, it atomically receives `false` and sets `lock=true`; it enters. All other calls return `true` and spin. The owner releases by storing `false`.

Atomic compare-and-swap (CAS):

```
int compare_and_swap(int *p, int expected, int newval) {
    int old = *p;
    if (old == expected) *p = newval;
    return old;
} // entire read/compare/conditional write is atomic

// Acquire a 0=free, 1=busy lock:
while (compare_and_swap(&lock, 0, 1) != 0)
    ;
/* CRITICAL SECTION */
lock = 0;
```

Atomic `swap`: exchanges two values unconditionally:

```
void swap(bool *a, bool *b) {
    bool t = *a;
    *a = *b;
    *b = t;
} // this whole exchange must be atomic in hardware
```

`swap` and `compare_and_swap` are **different instructions**: swap always exchanges; CAS writes only if the observed value matches `expected`. The Unit-3 PDF (p. 19) uses swap code under a “Compare and Swap” label; this answer distinguishes the two explicitly.

Comparison: TAS always sets the target to 1 and returns the old value; CAS sets the target to a new value **only when** the old value matches the expected value. Both are atomic building blocks for locks, but naïve busy-waiting loops can cause contention and starvation.

Q3. Give pseudocode for TestAndSet and Swap and prove critical-section conditions.

Answer:

Atomic `test_and_set` primitive:

```
bool test_and_set(bool *target) { // executed ATOMICALLY by hardware
    bool old = *target;
    *target = true;
    return old;
}
```

Spinlock using it:

```
bool lock = false;      // shared, initially unlocked
while (true) {
    while (test_and_set(&lock))
        ;               // busy wait
    /* CRITICAL SECTION */
    lock = false;       // atomic unlock/store
    /* REMAINDER SECTION */
}
```

If a process finds `lock=false`, it atomically receives `false` and sets `lock=true`; it enters. All other calls return `true` and spin. The owner releases by storing `false`.

Atomic compare-and-swap (CAS):

```
int compare_and_swap(int *p, int expected, int newval) {
    int old = *p;
    if (old == expected) *p = newval;
    return old;
} // entire read/compare/conditional write is atomic

// Acquire a 0=free, 1=busy lock:
while (compare_and_swap(&lock, 0, 1) != 0)
    ;
/* CRITICAL SECTION */
lock = 0;
```

Atomic `swap`: exchanges two values unconditionally:

```
void swap(bool *a, bool *b) {
    bool t = *a;
    *a = *b;
    *b = t;
} // this whole exchange must be atomic in hardware
```

`swap` and `compare_and_swap` are **different instructions**: swap always exchanges; CAS writes only if the observed value matches `expected`. The Unit-3 PDF (p. 19) uses swap code under a “Compare and Swap” label; this answer distinguishes the two explicitly.

Correctness analysis:

1. **Mutual exclusion:** each atomic TAS or swap-based acquisition admits only one contender when `lock` is free; no two processes can both acquire the same lock.
2. **Progress:** if the lock is free, at least one contending process can acquire it, assuming it is scheduled; a holder must eventually release it.
3. **Bounded waiting: not guaranteed by the plain TAS/swap loops.** A released lock may repeatedly be taken by one fast process while another spins, so the unmodified code is **not** a complete three-property critical-section solution.

To guarantee bounded waiting, use an additional waiting array/queue (e.g., a ticket lock or a bounded-waiting TAS design) and hand off permission in arrival order. A ticket lock gives each requester an increasing ticket and permits entry when `nowServing == myTicket`; the owner increments `nowServing` on exit. It preserves mutual exclusion and grants FIFO bounded waiting under eventual execution.

Q4. Explain with an algorithm how test_and_set satisfies critical-section requirements.

Answer:

Atomic `test_and_set` primitive:

```
bool test_and_set(bool *target) { // executed ATOMICALLY by hardware
    bool old = *target;
    *target = true;
    return old;
}
```

Spinlock using it:

```
bool lock = false;      // shared, initially unlocked
while (true) {
    while (test_and_set(&lock))
        ;               // busy wait
    /* CRITICAL SECTION */
    lock = false;       // atomic unlock/store
    /* REMAINDER SECTION */
}
```

If a process finds `lock=false`, it atomically receives `false` and sets `lock=true`; it enters. All other calls return `true` and spin. The owner releases by storing `false`.

Stepwise operation: initially lock=0. P1 executes TAS: returns 0 and changes lock→1, so P1 enters. P2 executes TAS: returns 1, so P2 remains in `while`. P1 finishes and writes lock=0. P2's next TAS can return 0 and change lock→1, so P2 enters. This proves **mutual exclusion** and enables progress when the lock becomes available. **Important exam caveat:** a basic TAS spinlock does **not** prove **bounded waiting** (Unit 3, p. 19); fairness requires extra logic.

Q5. Discuss hardware synchronization with pseudocode and examine the three conditions.

Answer:

Atomic compare-and-swap (CAS):

```
int compare_and_swap(int *p, int expected, int newval) {
    int old = *p;
    if (old == expected) *p = newval;
    return old;
} // entire read/compare/conditional write is atomic

// Acquire a 0=free, 1=busy lock:
while (compare_and_swap(&lock, 0, 1) != 0)
    ;
/* CRITICAL SECTION */
lock = 0;
```

Atomic `swap`: exchanges two values unconditionally:

```
void swap(bool *a, bool *b) {
    bool t = *a;
    *a = *b;
    *b = t;
} // this whole exchange must be atomic in hardware
```

`swap` and `compare_and_swap` are **different instructions**: swap always exchanges; CAS writes only if the observed value matches `expected`. The Unit-3 PDF (p. 19) uses swap code under a “Compare and Swap” label; this answer distinguishes the two explicitly.

Proof outline for atomic CAS spinlock:

1. The only entry is when `compare_and_swap(&lock,0,1)` returns 0. Because the test and write are atomic, two CPUs cannot both change the same free lock from 0 to 1 simultaneously. **Mutual exclusion holds**.
2. Once the current owner writes `lock=0`, an actively executing waiter can acquire it. Thus **progress is supported**, assuming the owner runs and releases.
3. CAS may repeatedly favor one contender; the waiting time of others has no fixed bound. Hence **bounded waiting is not guaranteed**. Use a FIFO/ticket lock or fair blocking mutex when bounded waiting is required.

Q6. What is the advantage of a semaphore over hardware-based busy-wait solutions?

Answer:

An OS-supported **blocking semaphore** can place a waiting process on a queue instead of spinning continuously on a hardware test-and-set/CAS lock. The CPU can execute other work while the process is asleep, especially useful for long waits. Semaphores also provide **counting** to manage several equivalent resources and ordering to coordinate producers/consumers. **Caution:** semaphores need atomic internal operations, often implemented using hardware primitives; semaphores can still deadlock or starve if used incorrectly, and some textbook versions themselves busy-wait.

Q7. Fill in the blank: The hardware implementation that provides mutual exclusion is _____.

Answer:

Answer: An **atomic lock instruction**, such as **`test_and_set`** (also **compare-and-swap** or atomic swap), can be used to implement mutual exclusion. The expected named primitive in Unit 3, pp. 16–18, is **`test_and_set`**.

Q8. List the advantages of spinlocks.

Answer:

1. **Very low uncontended overhead:** avoids process suspension and wake-up for short CS operations.
2. **Simple atomic construction:** can be built from TAS, CAS or atomic swap.
3. **Useful for very short kernel critical sections** when blocking is impossible or more expensive than waiting.
4. **No sleeping or wake-up transition** while the lock is held, reducing overhead under low contention.

5. **Useful on multicore systems** when another core is expected to release the lock very soon.

Limitation: spinning wastes CPU cycles, may increase memory-bus contention and does not necessarily provide FIFO fairness. Therefore use spinlocks only when the expected holding time is very short.

Section 9 — Semaphores and Mutex Locks (14 questions)

Q1. Define a semaphore and explain its need in process synchronization.

Answer:

A **semaphore** is an integer synchronization variable that is accessed only through two **atomic** operations: `wait(S)` (also called **P**) and `signal(S)` (also called **V**) (Unit 3, pp. 22–23).

```
wait(S):    acquire one available permit; if none, block/wait
signal(S):  release a permit, possibly waking one waiter
```

Conceptual counting definition:

```
wait(S) {
    while (S <= 0) ;    // textbook spinning variant
    S--;                 // test/decrement must together be atomic
}
signal(S) {
    S++;                 // increment must be atomic
}
```

Why needed: (1) enforce **mutual exclusion**, (2) order dependent tasks, (3) control access to **multiple instances** of a resource, and (4) avoid producer/consumer races. **Example:** a binary semaphore `mutex=1` protects a shared counter. Every process does `wait(mutex); counter++; signal(mutex);`, ensuring only one updates the counter at a time. In practical blocking semaphores, a waiter sleeps rather than repeatedly checking the value.

Q2. Name the two types of semaphores and give an example of each.

Answer:

1. Binary semaphore: count is either **0 or 1**, typically used for exclusive access. Example `mutex=1`; first `wait(mutex)` changes it to 0, so other threads must wait until `signal(mutex)` returns it to 1.

2. Counting semaphore: permits range from **0 to N** for N equivalent resource instances (an internal blocking implementation may represent waiting processes using a negative bookkeeping count). Example `printers=3`; at most three threads can hold a printer simultaneously. Calls `wait(printers)` decrease available permits; `signal(printers)` restores a permit. See Unit 3, p. 24.

Q3. Explain why semaphores are needed and implement a counting semaphore using binary semaphores.

Answer:

Need: protect shared data and represent the number of reusable resource instances without lost updates.

A correct conceptual implementation uses a **binary semaphore** `mutex=1` to protect the integer `permits` and a **private binary semaphore (initially 0) per blocked requester**. A FIFO queue stores blocked requesters. Private gates avoid lost wake-ups when several signals occur before sleepers actually block.

```
Shared: permits = K; mutex = binary(1); Q = FIFO empty

COUNT_WAIT():
    wait(mutex)
    if permits > 0:
        permits = permits - 1
        signal(mutex)
        return
    else:
        myGate = new binary_semaphore(0)
        enqueue(Q, myGate)
        signal(mutex)
        wait(myGate)        // sleep; a release transfers ownership
        return

COUNT_SIGNAL():
    wait(mutex)
    if Q is not empty:
        g = dequeue(Q)
        signal(g)           // pass released permit directly
    else:
        permits = permits + 1
    signal(mutex)
```

Step-by-step example K=2: (1) P1 calls COUNT_WAIT: permits 2→1; (2) P2 calls: 1→0; (3) P3 queues on its private gate; (4) P1 calls COUNT_SIGNAL, waking P3 and transferring P1's permit directly, while permits stays 0; (5) after P2 later releases with no waiter, permits becomes 1. **Why correct:** `mutex` makes queue/permit modification atomic, each `signal(myGate)` is retained until the matching wait succeeds, and FIFO queueing can give waiting order. This is an explanatory higher-level construction beyond the supplied slides, which give the definition and types but not a full binary-to-counting implementation.

Q4. List the drawbacks of semaphores.

Answer:

1. **Deadlock:** inconsistent wait order can make threads wait cyclically.
2. **Starvation:** an unfair semaphore queue may leave a waiter blocked indefinitely.
3. **Priority inversion:** a low-priority holder delays a higher-priority waiter (can be mitigated with priority inheritance).

4. **Programming errors:** missing `signal`, extra `signal` or incorrect operation order can corrupt synchronization.
5. **Busy waiting:** simple textbook implementations waste CPU time; blocking implementations need kernel/queue overhead.
6. **Difficult debugging:** timing-dependent races or deadlocks may be intermittent.

Unit 3, p. 41, explicitly lists **deadlock, starvation and priority inversion**.

Q5. Compare mutex locks and semaphores with examples.

Answer:

Property	Mutex lock	Semaphore
Purpose	Exclusive ownership of a critical section	Signaling, ordering, or resource-count management
Values	Locked / unlocked	Binary 0/1 or counting 0..N permits
Ownership	Normally only the owning thread unlocks	A different thread may `signal` a semaphore
Operations	`lock`, `unlock`	`wait` (P), `signal` (V)
Typical use	Protect shared counter, linked-list updates	Bound buffer slots, producer–consumer signaling, N printers
Overhead	Usually optimized for mutual exclusion; can block	Similar kernel/atomic costs; counting/signaling adds flexibility

Mutex example: `pthread_mutex_lock(&m); balance += 10; pthread_mutex_unlock(&m);`.

Semaphore example: `sem_t slots` initialized to 5; each producer does `sem_wait(&slots)` before using a slot and `sem_post(&slots)` after freeing one. **Key distinction:** a semaphore can coordinate different producer/consumer threads; a mutex protects an owned exclusive region.

Q6. Write a C program using a mutex to solve a critical-section problem; explain the execution.

Answer:

C program (POSIX threads; Linux: `gcc mutex.c -o mutex -pthread`):

```c
#include <stdio.h>
#include <pthread.h>

#define N 100000
long long counter = 0;
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;

void *worker(void *arg) {
    for (int i = 0; i < N; i++) {
        pthread_mutex_lock(&lock);     // entry section
        counter++;                     // critical section
        pthread_mutex_unlock(&lock);   // exit section
    }
    return NULL;
}

int main(void) {
    pthread_t t1, t2;
    pthread_create(&t1, NULL, worker, NULL);
    pthread_create(&t2, NULL, worker, NULL);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    printf("Final counter = %lld
", counter);
    pthread_mutex_destroy(&lock);
    return 0;
}
```

Explanation: (1) initialize `counter=0` and unlocked mutex; (2) create two worker threads; (3) each thread repeats 100000 increments; (4) before incrementing, it locks the mutex; (5) only the holder updates `counter`; (6) unlocking makes the mutex available to the other thread; (7) `pthread_join` waits for both threads; (8) final expected value = `2 × 100000 = 200000`. **Mutual exclusion** eliminates lost updates. This uses the Pthreads mutex API listed in Unit 3, pp. 30–31.

Q7. Show how incorrect semaphore use can cause deadlock.

Answer:

Given: two binary semaphores `S=1` and `Q=1`:

```
P1                      P2
wait(S);                wait(Q);
wait(Q);                wait(S);
// work                  // work
signal(Q);              signal(S);
signal(S);              signal(Q);
```

Deadlock trace: (1) P1 executes `wait(S)` → S=0; (2) P2 executes `wait(Q)` → Q=0; (3) P1 executes `wait(Q)` and blocks, waiting for P2; (4) P2 executes `wait(S)` and blocks, waiting for P1. Neither process reaches its signals. This creates a **circular wait**; neither can proceed.

Solution: enforce a single global acquisition order, e.g. both processes always `wait(S)` before `wait(Q)` and release in reverse order. The example is also given in Unit 3, p. 41.

Q8. What happens if a non-recursive mutex is locked twice by the same thread?

Answer:

Normally the second lock blocks and the thread deadlocks against itself, because the mutex is already held and the same thread cannot reach the unlock. In POSIX, behavior depends on the mutex type: `PTHREAD_MUTEX_ERRORCHECK` can return **EDEADLK**, while attempting to re-lock a default/normal non-recursive mutex is not a safe operation and may deadlock. A **recursive mutex** explicitly allows repeated locking by the same owner and requires the same number of unlocks. **Exam answer:** self-deadlock for a normal non-recursive mutex.

Q9. Counting semaphore initially 8: six P operations and four V operations. Find final value.

Answer:

Given: S_0=8, P=6, V=4. **Rule:** each completed P decreases count by 1 and each V increases it by 1.

$S_{final} = S_0 - P + V = 8 - 6 + 4 = 6$.

Answer: 6. Since the value never needs to become negative, the operations can complete without blocking if sequenced suitably.

Q10. Counting semaphore initially 7: twenty P and fifteen V operations. Find final value.

Answer:

Given: S_0=7, P=20, V=15.

$S_{final} = 7 - 20 + 15 = 2$.

Answer: 2. Completion of all twenty P operations is possible because the initial seven permits plus fifteen released permits provide 22 total permits. A blocking P may need to wait until an appropriate V executes; the final accounting remains 2.

Q11. Counting semaphore initially 7: after twenty P and x V, final value 5. Find x.

Answer:

Given: S_0=7, P=20, final S=5.

Use $S_{final} = S_0 - P + V$:

$5 = 7 - 20 + x$

$5 = -13 + x$

$x = 5 + 13 = 18$.

Answer: 18 V (signal) operations.

Q12. Three processes share semaphore S=2; P1 enters, P2 enters, P3 requests, and P2 exits. Find final semaphore value (negative means waiters).

Answer:

Here the question explicitly uses the **negative semaphore-value convention** (different from a non-negative "available permits" abstraction).

Step	Operation	S after operation	Explanation
Start	—	2	Two resources available
1	P1: wait(S)	1	P1 obtains first resource
2	P2: wait(S)	0	P2 obtains second resource
3	P3: wait(S)	−1	No free resource; P3 waits (one waiting process)
4	P2: signal(S)	0	Resource transferred to P3, which is awakened

Final answer: S = 0. No free permit remains after P3 is enabled to use the released instance.

Q13. Complete entry/exit sections for P1 and P2 updating X/Y under SX and SY without deadlock.

Answer:

Given: binary semaphores `SX=1`, `SY=1`; both variables must be updated as one consistent operation. Avoid deadlock by imposing the **same lock order SX → SY in both processes**, even though P2 updates Y first.

```
// Process P1
wait(SX);                   // entry: X lock first
wait(SY);                   // then Y lock
X = X + 10;
Y = Y - 20;
signal(SY);                 // exit: release in reverse order
signal(SX);

// Process P2
wait(SX);                   // SAME global lock order
wait(SY);
Y = Y + 20;
X = X - 10;
signal(SY);
signal(SX);
```

Reason: Neither process can hold SY and wait for SX, because both obtain SX first. Thus circular wait is impossible. At any moment only one process owns both locks and updates the shared variables. If P1 then P2 run, net change is X: `+10−10=0`, Y: `−20+20=0` relative to original values. Reversing only one process's acquisition order would risk deadlock.

Q14. Analyze code with a=1, b=0, count=0: each of n processes increments count under a, the nth signals b, then every process waits/signals b.

Answer:

Given:

```
wait(a);
count = count + 1;
if (count == n)
    signal(b);
signal(a);
wait(b);
signal(b);
// Code Section Q
```

Purpose: implements a **one-shot barrier** between Code Section P and Code Section Q: **no process may enter Q until all n processes have reached the barrier.**

Proof, step by step:

1. Initially `a=1`, `b=0`, `count=0`.
2. `wait(a)` serializes increments; therefore `count` correctly counts arrivals at the barrier.
3. The first n−1 processes increment count below n, release `a`, then block on `wait(b)` because b is 0.
4. The nth process increments count from n−1 to n and performs `signal(b)`, opening the gate.
5. One process passes `wait(b)`, immediately performs `signal(b)` and wakes the next; this **baton passing** repeats until all processes pass.
6. As every process reaches Q only after b has been opened, all n have completed P before any enters Q.

Answer: a reusable? **No, as written it is a one-time barrier**, because count is not reset and the gate remains logically open; extra phases require additional synchronization.

Section 10A — Classic Synchronization: Producer–Consumer / Bounded Buffer (3 questions)

Q1. Discuss the classical Producer–Consumer problem and write semaphore pseudocode.

Answer:

Problem: A buffer has N slots; a **producer** inserts an item and a **consumer** removes it. The producer must not insert if the buffer is full; the consumer must not remove if empty; concurrent modifications of the same buffer indices must not race (Unit 3, pp. 26–29).

Initialize three semaphores: `mutex = 1` (exclusive buffer access), `empty = N` (available free slots), `full = 0` (occupied slots).

```
// Producer
while (true) {
    item = produce_item();
    wait(empty);                    // reserve an empty slot
    wait(mutex);                    // lock the buffer
    buffer[in] = item;
    in = (in + 1) % N;
    signal(mutex);                  // unlock the buffer
    signal(full);                   // announce new item
}

// Consumer
while (true) {
    wait(full);                     // wait for an available item
    wait(mutex);                    // lock the buffer
    item = buffer[out];
    out = (out + 1) % N;
    signal(mutex);                  // unlock the buffer
    signal(empty);                  // announce freed slot
    consume_item(item);
}
```

Invariant: `empty + full = N` when accounting for all unreserved slots/items; during in-flight reservations/updates, the counters temporarily represent reserved capacities, not simply physical occupancy. **Important source note:** the Consumer columns on Unit-3 slides pp. 29 and 33 have incorrect repeated `signal(empty)` operations; the valid standard sequence is `signal(mutex)` **then** `signal(empty)` as shown above.

Stepwise example, N=3: Initially `(empty,full)=(3,0)`. P inserts A → after `wait(empty)` and `signal(full)`: `(2,1)`. P inserts B → `(1,2)`. C removes A → after `wait(full)` and `signal(empty)`: `(2,1)`. When full=0, the consumer blocks; when empty=0, the producer blocks. The mutex prevents simultaneous changes to buffer pointers.

Q2. Explain the bounded-buffer problem with an algorithm.

Answer:

Problem: A buffer has N slots; a **producer** inserts an item and a **consumer** removes it. The producer must not insert if the buffer is full; the consumer must not remove if empty; concurrent modifications of the same buffer indices must not race (Unit 3, pp. 26–29).

Initialize three semaphores: `mutex = 1` (exclusive buffer access), `empty = N` (available free slots), `full = 0` (occupied slots).

```
// Producer
while (true) {
    item = produce_item();
    wait(empty);                    // reserve an empty slot
    wait(mutex);                    // lock the buffer
    buffer[in] = item;
    in = (in + 1) % N;
    signal(mutex);                  // unlock the buffer
    signal(full);                   // announce new item
}

// Consumer
while (true) {
    wait(full);                     // wait for an available item
    wait(mutex);                    // lock the buffer
    item = buffer[out];
    out = (out + 1) % N;
    signal(mutex);                  // unlock the buffer
    signal(empty);                  // announce freed slot
    consume_item(item);
}
```

Invariant: `empty + full = N` when accounting for all unreserved slots/items; during in-flight reservations/updates, the counters temporarily represent reserved capacities, not simply physical occupancy. **Important source note:** the Consumer columns on Unit-3 slides pp. 29 and 33 have incorrect repeated `signal(empty)` operations; the valid standard sequence is `signal(mutex)` **then** `signal(empty)` as shown above.

Algorithm explanation: The producer (1) creates an item outside the CS, (2) reserves one free slot using `empty`, (3) acquires the mutex, (4) writes at `in` and advances `in` circularly, (5) releases the mutex, (6) signals `full`. The consumer (1) reserves an occupied slot with `full`, (2) locks the buffer, (3) reads `out` and advances it, (4) unlocks, (5) signals `empty`, and (6) uses the item. **No overwrite, underflow or data race is possible** if semaphore operations are atomic and every reservation is eventually matched by a release.

Q3. Give a solution to Producer–Consumer using synchronization primitives.

Answer:

Problem: A buffer has N slots; a **producer** inserts an item and a **consumer** removes it. The producer must not insert if the buffer is full; the consumer must not remove if empty; concurrent modifications of the same buffer indices must not race (Unit 3, pp. 26–29).

Initialize three semaphores: `mutex = 1` (exclusive buffer access), `empty = N` (available free slots), `full = 0` (occupied slots).

```
// Producer
while (true) {
    item = produce_item();
    wait(empty);                    // reserve an empty slot
    wait(mutex);                    // lock the buffer
    buffer[in] = item;
    in = (in + 1) % N;
    signal(mutex);                  // unlock the buffer
    signal(full);                   // announce new item
}

// Consumer
while (true) {
    wait(full);                     // wait for an available item
    wait(mutex);                    // lock the buffer
    item = buffer[out];
    out = (out + 1) % N;
    signal(mutex);                  // unlock the buffer
    signal(empty);                  // announce freed slot
    consume_item(item);
}
```

Invariant: `empty + full = N` when accounting for all unreserved slots/items; during in-flight reservations/updates, the counters temporarily represent reserved capacities, not simply physical occupancy. **Important source note:** the Consumer columns on Unit-3 slides pp. 29 and 33 have incorrect repeated `signal(empty)` operations; the valid standard sequence is `signal(mutex)` **then** `signal(empty)` as shown above.

C/POSIX correspondence (Unit 3, pp. 30–32): `wait(empty) = sem_wait(&empty)`, `signal(full) = sem_post(&full)`, and `wait(mutex)/signal(mutex)` may be realized by `pthread_mutex_lock(&mutex)` / `pthread_mutex_unlock(&mutex)`. `sem_init(&empty,0,N)`, `sem_init(&full,0,0)`, and `pthread_mutex_init(&mutex,NULL)` initialize the primitives; destroy them after all threads finish. **Ordering matters:** never acquire the mutex *before* `wait(empty)`/`wait(full)` or a blocked producer/consumer can hold the lock needed by its counterpart, causing deadlock.

Section 10B — Classic Synchronization: Readers–Writers (5 questions)

Q1. Discuss the Readers–Writers problem and present semaphore pseudocode.

Answer:

First Readers–Writers solution (reader preference), Unit 3, pp. 34–37: Many readers can read simultaneously, but a **writer** needs exclusive access.

Shared: `readcount=0`, binary semaphores `mutex=1` (protect count) and `wrt=1` (exclusive database access).

```
// Reader
while (true) {
    wait(mutex);
    readcount++;
    if (readcount == 1)
        wait(wrt);            // first reader blocks writers
    signal(mutex);

    /* READ DATABASE */

    wait(mutex);
    readcount--;
    if (readcount == 0)
        signal(wrt);          // last reader admits writer
    signal(mutex);
}

// Writer
while (true) {
    wait(wrt);
    /* WRITE DATABASE */
    signal(wrt);
}
```

Reasoning: the **first** reader locks `wrt`; subsequent readers increment `readcount` and proceed together. The **last** reader unlocks `wrt`. Writers hold `wrt` exclusively. **Caveat:** a steady arrival of readers can starve writers (reader preference does **not** ensure fairness).

Permitted combinations: `reader + reader` is safe; `reader + writer` is forbidden; `writer + writer` is forbidden. **Example:** R1 enters first (`readcount=1`, locks wrt), R2 enters (`readcount=2`, shares reading), W1 waits on wrt; after R1 exits count=1 and writer still waits; R2 exits count=0 and signals wrt; W1 finally writes.

Q2. Write routines to solve the Readers–Writers synchronization problem.

Answer:

First Readers–Writers solution (reader preference), Unit 3, pp. 34–37: Many readers can read simultaneously, but a **writer** needs exclusive access.

Shared: `readcount=0`, binary semaphores `mutex=1` (protect count) and `wrt=1` (exclusive database access).

```
// Reader
while (true) {
    wait(mutex);
    readcount++;
    if (readcount == 1)
        wait(wrt);            // first reader blocks writers
    signal(mutex);

    /* READ DATABASE */

    wait(mutex);
    readcount--;
    if (readcount == 0)
        signal(wrt);          // last reader admits writer
    signal(mutex);
}

// Writer
while (true) {
    wait(wrt);
    /* WRITE DATABASE */
    signal(wrt);
}
```

Reasoning: the **first** reader locks `wrt`; subsequent readers increment `readcount` and proceed together. The **last** reader unlocks `wrt`. Writers hold `wrt` exclusively. **Caveat:** a steady arrival of readers can starve writers (reader preference does **not** ensure fairness).

Functions/routines: The reader entry routine acquires `mutex`, increments `readcount`, and if it is the first reader obtains `wrt`; reader exit decrements `readcount` and releases `wrt` if the last. Writer entry waits on `wrt`; writer exit signals `wrt`. All these sequences are shown in the code. **Safety:** writing occurs only when there are no readers or other writers.

Q3. Write the structure of a reader process in the First Readers–Writers problem.

Answer:

Shared state: `int readcount=0; semaphore mutex=1, wrt=1;`. The **required reader routine** is:

```
while (true) {
    wait(mutex);
    readcount = readcount + 1;
    if (readcount == 1)
        wait(wrt);
    signal(mutex);

    /* perform reading */

    wait(mutex);
    readcount = readcount - 1;
    if (readcount == 0)
        signal(wrt);
    signal(mutex);
}
```

Explain each step: lock `mutex` before changing count; first reader excludes writers; unlock mutex so additional readers may enter; read concurrently; lock mutex on exit; last reader enables a writer; unlock mutex. Note that **this first solution may starve writers** under continuous reader arrivals (Unit 3, pp. 36–37).

Q4. Develop a Readers–Writers semaphore solution and explain fairness and synchronization.

Answer:

Requirement: allow multiple concurrent readers, only one writer, and avoid indefinite reader/writer overtaking.

Fair (no-barging) variant: Introduce an additional binary **serviceQueue / turnstile** semaphore, assumed to serve blocked processes FIFO, to stop newly arriving readers from indefinitely overtaking a writer. Keep `resource=1`, `rmutex=1`, `serviceQueue=1`, `readcount=0`.

```
// Fair reader
wait(serviceQueue);           // line up fairly
wait(rmutex);
readcount++;
if (readcount == 1)
    wait(resource);           // first reader locks database
signal(serviceQueue);
signal(rmutex);

/* READ */

wait(rmutex);
readcount--;
if (readcount == 0)
    signal(resource);         // last reader releases database
signal(rmutex);

// Fair writer
wait(serviceQueue);           // join the common FIFO line
wait(resource);               // exclusive database access
signal(serviceQueue);
/* WRITE */
signal(resource);
```

Fairness explanation: Once a writer reaches the turnstile, later readers cannot pass it; earlier readers finish and release the database so the writer proceeds. **Assumption:** FIFO (or otherwise starvation-free) waiter selection and eventual termination of readers/writers; the simple Unit-3 lecture algorithm omits the turnstile and therefore does not guarantee fairness. This is an explicit extension to answer the fairness question.

Correctness: `resource` protects writer exclusivity; `readcount` with `rmutex` lets a first reader acquire and last reader release `resource`; `serviceQueue` serializes the admission decision so later arrivals do not barge ahead. The solution provides mutual exclusion of writers and shared access by readers. With a FIFO sem implementation and finite critical sections, waiting is bounded by earlier requests, preventing starvation.

Q5. Explain the Readers–Writers problem with an algorithm.

Answer:

First Readers–Writers solution (reader preference), Unit 3, pp. 34–37: Many readers can read simultaneously, but a **writer** needs exclusive access.

Shared: `readcount=0`, binary semaphores `mutex=1` (protect count) and `wrt=1` (exclusive database access).

```
// Reader
while (true) {
    wait(mutex);
    readcount++;
    if (readcount == 1)
        wait(wrt);            // first reader blocks writers
    signal(mutex);

    /* READ DATABASE */

    wait(mutex);
    readcount--;
    if (readcount == 0)
        signal(wrt);          // last reader admits writer
    signal(mutex);
}

// Writer
while (true) {
    wait(wrt);
    /* WRITE DATABASE */
    signal(wrt);
}
```

Reasoning: the **first** reader locks `wrt`; subsequent readers increment `readcount` and proceed together. The **last** reader unlocks `wrt`. Writers hold `wrt` exclusively. **Caveat:** a steady arrival of readers can starve writers (reader preference does **not** ensure fairness).

Algorithm trace: R1 starts: readcount `0→1`, `wrt:1→0`. R2 starts: count `1→2`, no second wait on wrt. W1 arrives: waits because wrt=0. R1 leaves: count `2→1`, wrt remains 0. R2 leaves: count `1→0`, signals wrt to 1. W1 acquires wrt and writes alone, then releases it. **Important:** use the **fair variant** with `serviceQueue` if the question requires preventing starvation; the simple textbook algorithm only guarantees reader–writer mutual exclusion.

Section 10C — Classic Synchronization: Dining Philosophers (6 questions)

Q1. Describe the Dining Philosophers problem in detail.

Answer:

Problem: Five philosophers sit around a circular table, each alternating between **thinking** and **eating**. Five forks/chopsticks lie between adjacent philosophers. Philosopher i requires

both adjacent forks `fork[i]` and `fork[(i+1)%5]` to eat, and releases both afterward (Unit 3, pp. 38–40).

Simple, unsafe code:

```
wait(fork[i]);
wait(fork[(i+1)%5]);
eat();
signal(fork[(i+1)%5]);
signal(fork[i]);
```

Deadlock trace: If all five pick up their left fork first, every fork is held by one philosopher. Each then waits for the right fork held by a neighbor. All five wait forever in a circular chain. **Possible remedies from Unit 3, p. 39:** (i) permit at most four philosophers to try at a time, (ii) make acquisition of both forks atomic, or (iii) asymmetric order (odd/even philosophers pick in opposite orders). Starvation requires a separate fairness argument.

Properties to ensure: (1) **mutual exclusion** on each fork, (2) **deadlock freedom**, and ideally (3) **no starvation** of a philosopher. **Why it matters:** it models allocation of multiple shared resources, lock-order bugs and circular waits in OS and database systems. **Approach:** restrict simultaneous contenders, impose resource ordering, or grant both forks together.

Q2. Implement Dining Philosophers using monitors and discuss its use.

Answer:

Monitor-based dining-philosophers solution (standard extension; monitor code is not included in the provided Unit-3 slides):

```
monitor DiningPhilosophers {
    enum {THINKING, HUNGRY, EATING} state[5];
    condition self[5];

    LEFT(i)  = (i + 4) % 5
    RIGHT(i) = (i + 1) % 5

    test(i):
        if state[i] == HUNGRY and
           state[LEFT(i)] != EATING and
           state[RIGHT(i)] != EATING:
            state[i] = EATING
            signal(self[i])

    pickup(i):
        state[i] = HUNGRY
        test(i)
        if state[i] != EATING:
            wait(self[i])     // atomically release monitor and sleep

    putdown(i):
        state[i] = THINKING
        test(LEFT(i))
        test(RIGHT(i))
}

philosopher(i):
    while true:
        think()
        DiningPhilosophers.pickup(i)
        eat()
        DiningPhilosophers.putdown(i)
```

Reasoning: the monitor serializes changes to the state array, and `test(i)` grants permission only if neither neighbor is eating. Hence adjacent philosophers cannot eat simultaneously. All five may become hungry but no one holds one fork while waiting for the other, so the circular-fork deadlock is avoided. **Caveat:** this common monitor may starve a philosopher if neighbors repeatedly win; a FIFO queue/ticket fairness policy is needed for a starvation-free version.

Use: monitors bundle a shared state with procedures executed under implicit mutual exclusion. Condition variables release the monitor on `wait`, allowing neighbors to finish and awaken a hungry philosopher. **Example:** when P0 is eating, P1 and P4 fail `test` because P0 is their neighbor. P2 may eat if its neighbors are not eating. On P0’s `putdown`, the monitor checks both neighbors and signals whichever can eat. A monitor is easier to reason about than managing several fork locks manually.

Q3. Write C routines for a deadlock-free solution to Dining Philosophers.

Answer:

Deadlock-free semaphore algorithm (room limit):

```
semaphore room = 4;
semaphore fork[5] = {1, 1, 1, 1, 1};

void philosopher(int i) {
    while (true) {
        think();
        wait(room);             // at most 4 contenders
        wait(fork[i]);
        wait(fork[(i + 1) % 5]);
        eat();
        signal(fork[(i + 1) % 5]);
        signal(fork[i]);
        signal(room);
    }
}
```

Why no deadlock: When at most four philosophers may compete, at least one of the five forks is not held by a participating philosopher during any configuration of “each holds one fork,” so the complete 5-way circular wait cannot occur. Some participant can obtain a second fork and finish. **Caveat:** this proves deadlock freedom, **not unconditional starvation freedom**, if the semaphore implementation is unfair.

C routines corresponding to the pseudocode:

```
#include <semaphore.h>
#define N 5
sem_t forks[N], room;

void take_forks(int i) {
    sem_wait(&room);
    sem_wait(&forks[i]);
    sem_wait(&forks[(i + 1) % N]);
}
void put_forks(int i) {
    sem_post(&forks[(i + 1) % N]);
    sem_post(&forks[i]);
    sem_post(&room);
}
/* In main: sem_init(&room,0,4); initialize each fork to 1. */
```

Explanation: philosophers invoke `take_forks(i)` before `eat()`, then `put_forks(i)` after finishing. `room=4` removes the possibility that all five simultaneously hold their left fork and block. This is deadlock-free provided holders eventually finish eating.

Q4. Write a C program to solve Dining Philosophers using semaphores.

Answer:

Complete C / Pthreads / POSIX semaphore program (`gcc dining.c -pthread -o dining`):

```c
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>

#define N 5
#define ROUNDS 2
sem_t fork_sem[N], room;

void *philosopher(void *arg) {
    int i = *(int *)arg;
    for (int round = 0; round < ROUNDS; round++) {
        printf("P%d thinking
", i);
        sem_wait(&room);                        // max 4 contenders
        sem_wait(&fork_sem[i]);
        sem_wait(&fork_sem[(i + 1) % N]);

        printf("P%d eating (round %d)
", i, round + 1);
        /* eating work */

        sem_post(&fork_sem[(i + 1) % N]);
        sem_post(&fork_sem[i]);
        sem_post(&room);
    }
    return NULL;
}

int main(void) {
    pthread_t t[N];
    int id[N];
    sem_init(&room, 0, N - 1);
    for (int i = 0; i < N; i++)
        sem_init(&fork_sem[i], 0, 1);

    for (int i = 0; i < N; i++) {
        id[i] = i;
        pthread_create(&t[i], NULL, philosopher, &id[i]);
    }
    for (int i = 0; i < N; i++)
        pthread_join(t[i], NULL);

    for (int i = 0; i < N; i++)
        sem_destroy(&fork_sem[i]);
    sem_destroy(&room);
    return 0;
}
```

Steps: (1) create five fork semaphores, each 1; (2) initialize `room=4`; (3) create five philosopher threads; (4) each takes the room permit and both forks before eating; (5) after eating, it releases both forks and room; (6) join and clean up. **Why correct:** neighbors cannot use the same fork concurrently; at most four contenders eliminates complete circular deadlock. **Output order may differ** on each run because scheduling is concurrent. The room-limit method is taken from Unit 3, p. 39; complete runnable C is supplemental.

Q5. Design a solution guaranteed free from both deadlock and starvation.

Answer:

Provably deadlock-free and starvation-free (under finite eating and fair scheduling): FIFO waiter/host monitor. This conservative solution allows **one philosopher to eat at a time**, ensuring both forks are available. It sacrifices concurrency in exchange for an easy fairness proof.

```
Shared in monitor:
    nextTicket = 0
    servingTicket = 0
    condition turnChanged

pickup(i):
    myTicket = nextTicket
    nextTicket = nextTicket + 1
    while myTicket != servingTicket:
        wait(turnChanged)     // release monitor while sleeping
    // i now owns the dining permission for both forks

putdown(i):
    servingTicket = servingTicket + 1
    broadcast(turnChanged)

Philosopher i:
    while true:
        think()
        pickup(i)
        eat()                 // keep exclusive permission until finished
        putdown(i)
```

Mutual exclusion: Only the philosopher whose ticket equals `servingTicket` may eat. **Deadlock freedom:** no philosopher waits while owning a fork or a different needed lock; the current ticket-holder can eat and release. **Starvation freedom:** all requests receive monotonically increasing FIFO tickets; a philosopher is preceded by only finitely many older tickets. If every meal eventually finishes and threads are eventually scheduled, the ticket is eventually served. **Trade-off:** concurrency is limited (one eater at a time), but it satisfies the two requested liveness properties.

Note: a room semaphore set to 4 prevents deadlock but by itself does **not** guarantee starvation freedom. The ticket-host approach explicitly imposes FIFO admission; this is why it satisfies both requirements. For higher concurrency, a more sophisticated fair central waiter can grant non-adjacent philosophers together while preserving FIFO progress.

Q6. Explain the Dining Philosophers problem with an algorithm.

Answer:

Problem: Five philosophers sit around a circular table, each alternating between **thinking** and **eating**. Five forks/chopsticks lie between adjacent philosophers. Philosopher i requires **both** adjacent forks `fork[i]` and `fork[(i+1)%5]` to eat, and releases both afterward (Unit 3, pp. 38–40).

Simple, unsafe code:

```
wait(fork[i]);
wait(fork[(i+1)%5]);
eat();
signal(fork[(i+1)%5]);
signal(fork[i]);
```

Deadlock trace: If all five pick up their left fork first, every fork is held by one philosopher. Each then waits for the right fork held by a neighbor. All five wait forever in a circular chain. **Possible remedies from Unit 3, p. 39:** (i) permit at most four philosophers to try at a time, (ii) make acquisition of both forks atomic, or (iii) asymmetric order (odd/even philosophers pick in opposite orders). Starvation requires a separate fairness argument.

Deadlock-free semaphore algorithm (room limit):

```
semaphore room = 4;
semaphore fork[5] = {1, 1, 1, 1, 1};

void philosopher(int i) {
    while (true) {
        think();
        wait(room);             // at most 4 contenders
        wait(fork[i]);
        wait(fork[(i + 1) % 5]);
        eat();
        signal(fork[(i + 1) % 5]);
        signal(fork[i]);
        signal(room);
    }
}
```

Why no deadlock: When at most four philosophers may compete, at least one of the five forks is not held by a participating philosopher during any configuration of “each holds one fork,” so the complete 5-way circular wait cannot occur. Some participant can obtain a second fork and finish. **Caveat:** this proves deadlock freedom, **not unconditional starvation freedom**, if the semaphore implementation is unfair.

Algorithm walkthrough: P0 requests room, fork0 and fork1, eats, then releases fork1, fork0 and room. P1 may hold fork1 only after P0 releases it; while one philosopher is eating, neither immediate neighbor can eat, but non-adjacent philosophers may eat concurrently. Since only four may compete for forks, a circular wait involving all five is broken. **Answer:** the modified semaphore algorithm provides mutual exclusion and avoids deadlock; fairness requires FIFO admission or another starvation-prevention rule.

Answer coverage and reference note

Coverage checklist: Section 1: 12; Section 2: 4; Section 3: 19; Section 4: 3; Section 5: 4; Section 6: 5; Section 7: 5; Section 8: 8; Section 9: 14; Section 10A: 3; Section 10B: 5; Section 10C: 6. **Total: 88 questions.**

Primary references: `unit 2.pdf` (RVCE Operating Systems Unit-2 slides, 54 pages, especially pp. 22–54) and `unit 3.pdf` (RVCE Operating Systems Unit-3 slides, 41 pages, especially pp. 2–41). Supplementary material is used only for questions that ask beyond what the slides explain (RMS/EDF, precise CAS distinction, fair Readers–Writers, monitor/FIFO Dining Philosophers, runnable C code). Inconsistent source pseudocode is identified where needed rather than copied uncritically.